

FastAPI

Python API



Ako Začneme?

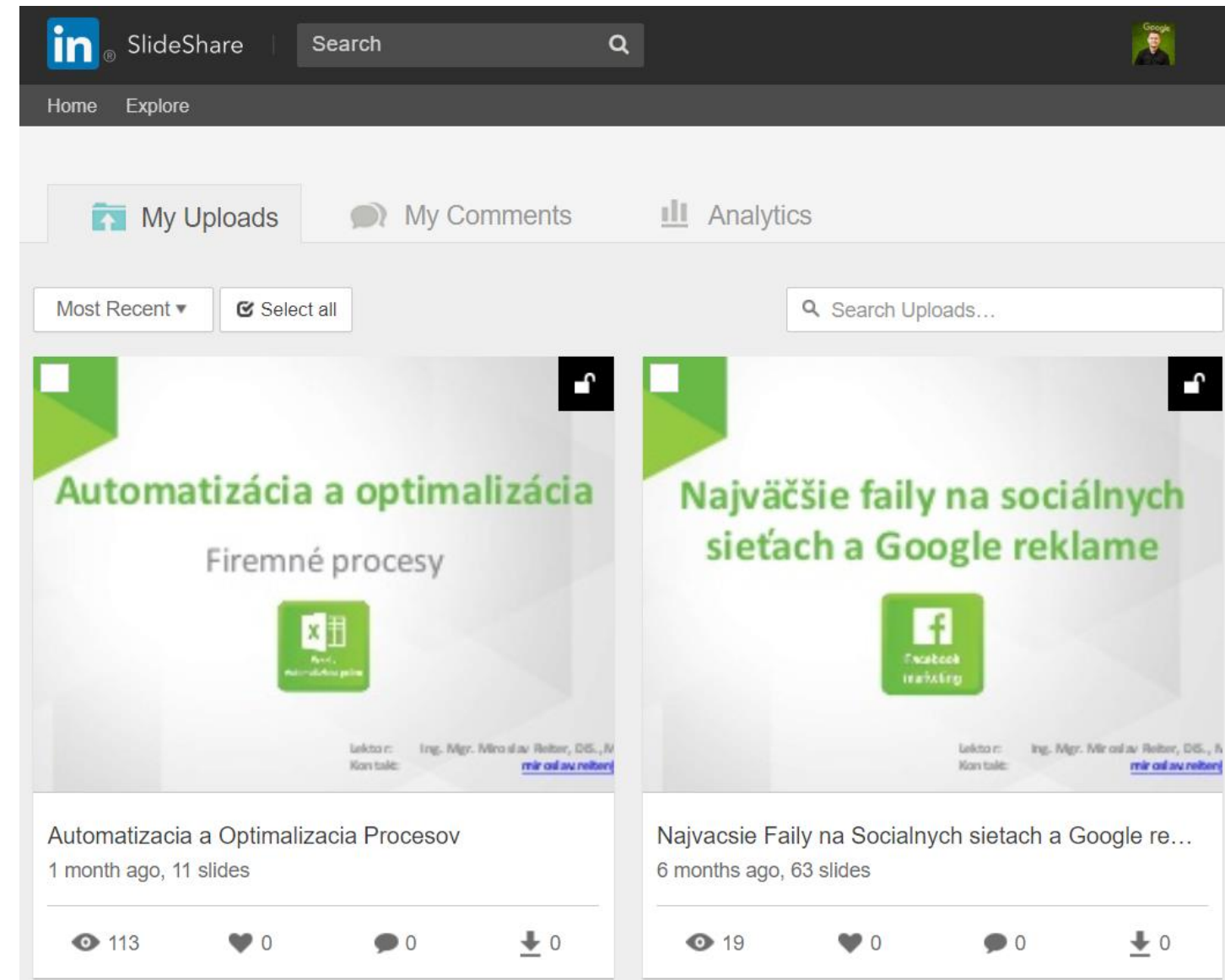
2

1. Pridajte si ma na **LinkedIn**
www.linkedin.com/in/miroslav-reiter

2. Stiahnite si Cvičný NTB Súbor
github.com/miroslav-reiter/Kurzy_SAV_Analytika_Python_R

3. Prezentácia po prednáške
<https://github.com/miroslav-reiter/>

4. Videá na YouTube
www.youtube.com/@VITA-Academy





Miroslav Reiter

miroslav-reiter

Founder of VITA Academy ★ Microsoft Certified Trainer ★ Google, Android Certified Trainer ★ ISTQB Trainer

Edit profile

81 followers · 2 following

VITA Academy

Bratislava

miroslav.reiter@it-academy.sk

www.vita.sk

https://orcid.org/0000-0003-1804-651X

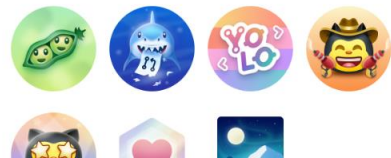
@VITA_Academy_SK

in/miroslav-reiter

VitaAcademySK

@VITA-Academy

Achievements



miroslav-reiter / README.md

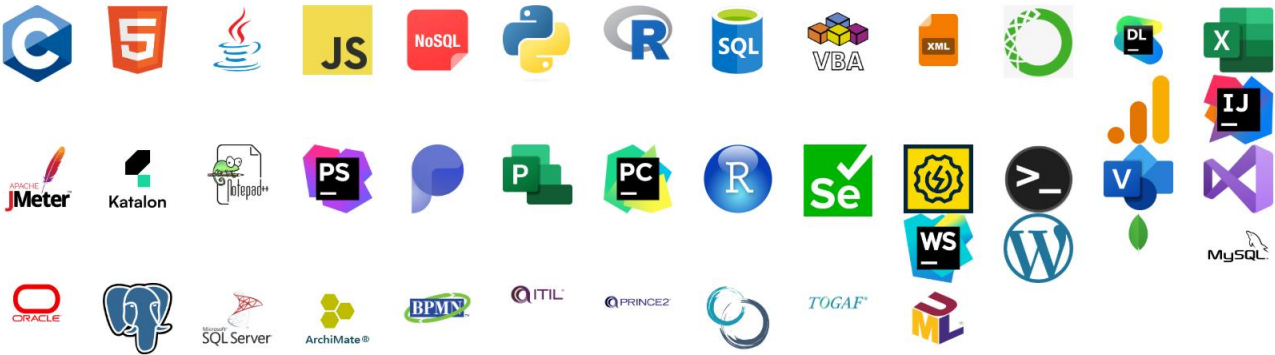
About me

- 👋 Hi, I am @Miroslav-Reiter
- 👤 I am a Google Certified Trainer GCT, Microsoft Certified Trainer MCT, Microsoft Most Valuable Professional MVP, ISTQB/GASQB Trainer, PRINCE2/ITIL4/ArchiMate/TOGAF/UML/BPMN/Scrum Trainer
- 🔧 My preferred programming languages are Java ☕, Python 🐍, JavaScript (Google Apps Script), VBA and R
- 🎓 I teach programming and how to use IT technologies effectively at [VITA](#) and [IT Academy](#) My online courses can be bought directly at [VITA.sk](#). I recommend buying an annual subscription with all courses
Online Akreditovane Kurzy a Skolenia [VITA](#)
- 🗣️ I frequently speak at conferences and workshops. Mostly about IT (programming, automation, certification, testing, enterprise architecture and modeling), **management** (Project management, PRINCE2, ITIL4), **education** and **online marketing** with ads (Google Ads, Google Analytics, Copywriting, Social media, YouTube)
- 📺 I regularly upload a new videos to [YouTube channel Miroslav Reiter - VITA Academy](#)
[YouTube IT Academy](#) 9.5k
- ⚡ Fun fact: I have graduated from 14 different Universities. I am currently studying at 4 other Universities, 3 of which are PhDs. Check out my [LinkedIn](#) profile

My Socials



Languages, Tools and Frameworks



Kurzy_SAV_DataScience_Python_BI_AI_ML

Public

Unpin

Unwatch16

Fork23

Starred22

main

1 Branch

0 Tags

Go to file

Add file

<> Code

miroslav-reiter Update README.md	0ec8ff4 · now	🕒 217 Commits
Datasety	Add files via upload	10 months ago
Prezentacie	Add files via upload	2 weeks ago
Zdrojove-Kody	Delete Zdrojove-Kody/Python-3-OOP2/a	2 months ago
Cvicenia-Jemny-Uvod-do-AI-ChatGPT-2024.pdf	Add files via upload	6 months ago
Klavesnica-Python.pdf	Add files via upload	3 years ago
README.md	Update README.md	now

README

Kurzy SAV DataScience, Python, R, Julia, BI, AI/ML, ChatGPT

Materiály, Zdrojové Kódy a Projekty, Prezentácie ku kurzom SAV DataScience, Python, OOP, R, BI, Analytika

Python je interpretovaný, interaktívny, open-source programovací jazyk.

Python beží na mnohých variantoch Unixu, na Macu a Windowse (súčasťou kurzu bude inštalácia na vašom systéme). Pre absolvovanie kurzu je potrebné mať k dispozícii vlastný notebook (s ľubovoľným operačným systémom podporujúcim Python).

R je programovací jazyk a softvérové prostredie pre štatistickú analýzu a vizualizáciu. R je voľne dostupný pod GNU licenciou a existujú verzie pre mnohé operačné systémy ako Linux, Windows a Mac.

ChatGPT je pokročilý jazykový model umelej inteligencie vyvinutý spoločnosťou OpenAI. Je to variant modelu GPT (Generative Pre-trained Transformer), ktorá je špeciálne navrhnutá na generovanie textu a interakciu v dialógovej

About

Materiály, Zdrojové Kódy, Prezentácie ku kurzom SAV Python, OOP, R, BI, Data Science

python

jupyter

numpy

oop

jupyter-notebook

pandas

python3

matplotlib

sav

beatifulsoup

reiter

matplotlib-pyplot

Readme

Activity

22 stars

16 watching

23 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Languages

Jupyter Notebook96.5%

HTML3.3%

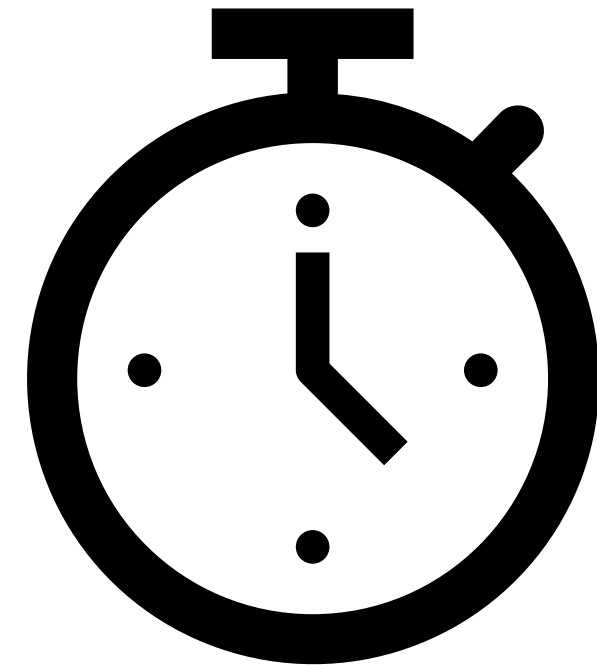
Other0.2%

Suggested workflows

Úvodné Informácie

- Časový rozvrh (9:00-13:30)
- Programujeme a testujeme (50 min)
- Prestávky (10 min)
- Obedová prestávka
- Mobilné telefóny a zariadenia

- Priprav si otázky a rovno sa pýtaj
- Interaktívna forma



O mne - Miroslav Reiter

6

40000+
klientov a
1000+ firiem

IT Architekt
Programátor
Manažér

Microsoft
Google
ISTQB tréner

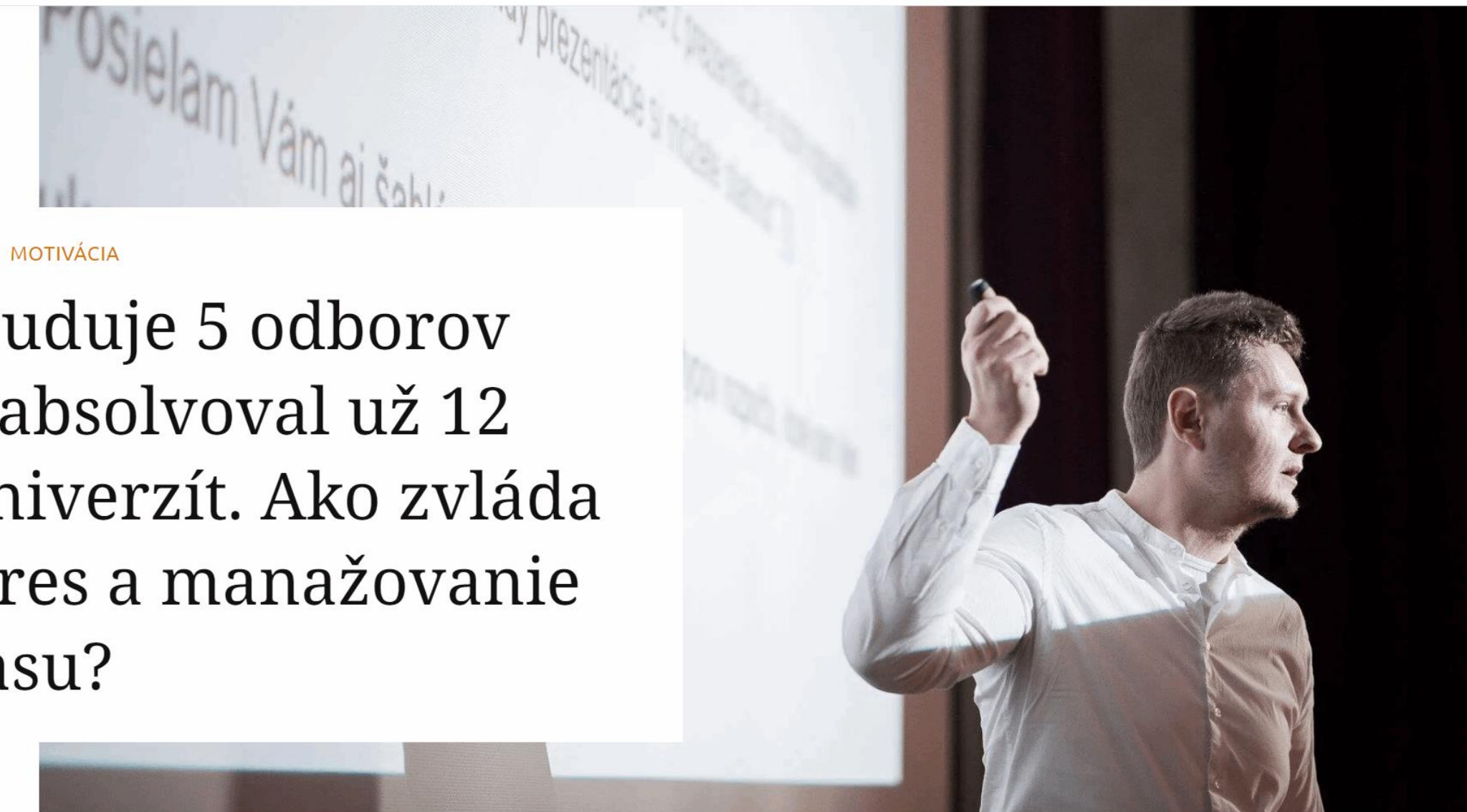
134 certifikácii

151 príručiek
a publikácií

14 škôl

62 projektov

Vlastná firma



— MOTIVÁCIA

Študuje 5 odborov a absolvoval už 12 univerzít. Ako zvláda stres a manažovanie času?

Foto: Jakub Kovalík pre FMK UCM | Miroslav Reiter na prednáške Grow with Google na FMK UCM.



Nikola Kotlárková

19. júl 2022 · 8 min. čítania





Miroslav Reiter

1. PhDr. VŠM (Podnikovný manažment)
2. Ing. STU FEI (Aplikovaná informatika)
3. Mgr. UK FM (Strategický manažment a marketing)
4. Mgr. VŠM (Manažérstvo kvality)
5. Mgr. VŠEMVŠ (Verejná správa)
6. Mgr. DTI (Učiteľstvo ekonomických predmetov)
7. DiS. AMOS (Cestovný ruch)
8. MBA LIGS (Executive management)
9. DBA Humanum (IT manažment)
10. MPA IES (Verejná správa a samospráva krajov)
11. MSc. Humanum (Bezpečnosť inf. systémov)
12. Ing. Paed. IGIP STU
13. Mgr. PEVŠ (Bezpečnosť informačných systémov)
14. RNDR. PEVŠ (Bezpečnosť informačných systémov)



FAKULTA MANAGEMENTU

Univerzita Komenského
v Bratislave



IT ACADEMY

**DIGITÁLNA
UNIVERZITA**



**VITA
ACADEMY**



Miroslav Reiter

 Slovakia

 IT Academy s.r.o.

Profile Events

Headline

 IT Architect and Programmer  Hard worker 
Lecturer and Certified Trainer

Biography

 I'm a hard worker, intellectual and joker. I love learning and teaching as well. My main objective is to teach people and improve their IT knowledge. To create truly practical knowledge necessary for life and present it in an interesting way. I don't like snake charmers and people who cannot...

✓ Read more

Award Category

M365

Technology Area

Visio, Excel

Languages

English, Slovak

Social



Most Valuable Professionals

High Impact Activities

This community leader has not added a high impact activity yet.

 [Domov](#) > [Registre](#) > [Znalci](#) > PhDr. Ing. Mgr. Miroslav Reiter

Znalec

PhDr. Ing. Mgr. Miroslav Reiter

Evidenčné číslo: 915864

Miesto výkonu činnosti

Tomášikova 50
83104 Bratislava
Slovenská republika

[Zobraziť na mape](#)

Kontaktné údaje

Mobil: +421 908 163 084
E-mail: znalec@it-academy.sk

Odbory a odvetvia

Odbor / Odvetvie	História	Stav
100000 - Elektrotechnika		
100400 - Riadiaca technika, výpočtová technika (hardware)	14.02.2023 - Zápis	AKTÍVNY
100800 - Nosiče zvukových a zvukovoobrazových záznamov	14.02.2023 - Zápis	AKTÍVNY
100900 - Počítačové programy (software)	14.02.2023 - Zápis	AKTÍVNY
101000 - Bezpečnosť a ochrana informačných systémov	14.02.2023 - Zápis	AKTÍVNY

Vyberte si online kurz

Naučte sa programovať, tvoriť webstránky a grafiku, manažovať alebo sa zamerajte na osobný rozvoj. Všetko jednoducho vďaka našim online kurzom z pohodlia domova.

Ročné Predplatné na všetky Online Kurzy

~~2290 €~~

490 €

Prístup pre Vás do všetkých Aktuálnych aj
Pripravovaných Online Kurzov

12 mesačná platnosť

Kúpiť teraz

[Zistiť viac](#)



560 kurzov v ponuke



Zábavné online lekcie



Akreditované kurzy



13 rokov skúseností



Certifikovaní
profesionálni lektori

Odporúčame Kurzy špeciálne pre vás



Online kurz SAP I.
Začiatočník
298,00€ ~~398,00€~~



Balík SAP Profesionál
998,80€ ~~998,80€~~



Online kurz ChatGPT a DALL-E AI
186,00€ ~~254,00€~~

SQL

Online kurz SQL I.
Začiatočník
196,00€ ~~276,00€~~



Program MPA Manažér
Štátnej a Verejnej Správy
1 940,00€ ~~2 540,00€~~



Online kurz Lektor
(Akreditovaný Kurz Lektor)
238,00€ ~~296,00€~~

Chat



Comments4 ; **Online kurz Python** - Cyklus While, break a continue. Miroslav
Reiter - VITA Academy · 156 views ; Online kurz Jazyk Julia - Ako ...

www.youtube.com › watch

Ako začať programovať v Pythone? - Online kurz Python a ...



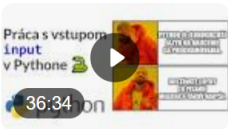
... Python I. Začiatočník: ➔ <https://www.it-academy.sk/kurz/python/> Online kurz Python ... Online #Kurzy #Školenia #Programovanie #Python...

YouTube · Miroslav Reiter - VITA Academy · 10. 11. 2021

16 kľúčových momentov v tomto videu

www.youtube.com › watch

Online kurz Python Základy - Práca s Vstupom (input) v ...



... Online #Kurzy #Školenia #Programovanie #Jetbrains #Datalore #Jupyter.

... <https://www.vita.sk/on...> Akreditovaný online kurz Programovanie ...

YouTube · Miroslav Reiter - VITA Academy · 25. 8. 2023



4 kľúčové momenty v tomto videu

www.youtube.com › watch

Online Kurz Python I. Začiatočník - Ukážka - IDE, Pycharm ...



... kurz/python/ Online kurz Python I. Začiatočník je pre teba vhodný, ak si

... Online #Kurzy #Školenia #Programovanie #Jazyk #Python #Idle.

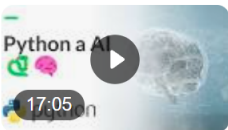
YouTube · Miroslav Reiter - VITA Academy · 2. 3. 2020



10 kľúčových momentov v tomto videu

www.youtube.com › watch

Online kurz Python - Používanie AI pri Programovaní v ...



... Online kurz Python - Používanie AI pri Programovaní v Pythone a

Generovanie Zdrojových Kódov Akreditovaný online kurz Programovanie...

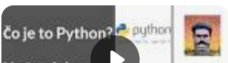
YouTube · Miroslav Reiter - VITA Academy · 25. 9. 2023



4 kľúčové momenty v tomto videu

www.youtube.com › watch

Online kurz Python - Čo je to Python? - Na čo a kde sa Python ...



Comments3 · Online Kurz Python Objektové Programovanie - Čo je to Polymorfizmus v Pythone? · Online Kurz Úvod do Programovania - Ukážk...



Luigi, Mário 
a Yoshi

Čo Robíte?

16

1. Študent/Učiteľ

2. Zamestnanec

3. Podnikateľ

4. Nezamestnaný/materská

5. Dievča pre všetko



Vaše Ciele a Očakávania 17

1. Doplniť si znalosti z jazyka Python

2. Naučiť sa knižnicu Keras (TensorFlow)

3. Vytvoriť a natrénovať jednoduchý model NS

4. Pochopiť štruktúru NS v Keras

5. Práca s modelmi a výsledkami

Zábava je v zaručená v každom bode 😊



Ako začať s FastAPI 2

AKREDITOVANÝ KURZ



FastAPI





Čo sa naučíme?

1. Čo je to API, FastAPI, Pydantic?
2. Aké sú kľúčové komponenty?
3. Aké sú hlavné vlastnosti?
4. Čo si treba nainštalovať?
5. Ako začať používať FastAPI
6. Ako sa pracuje s portami?
7. Profit

FastAPI Framework v Pythone

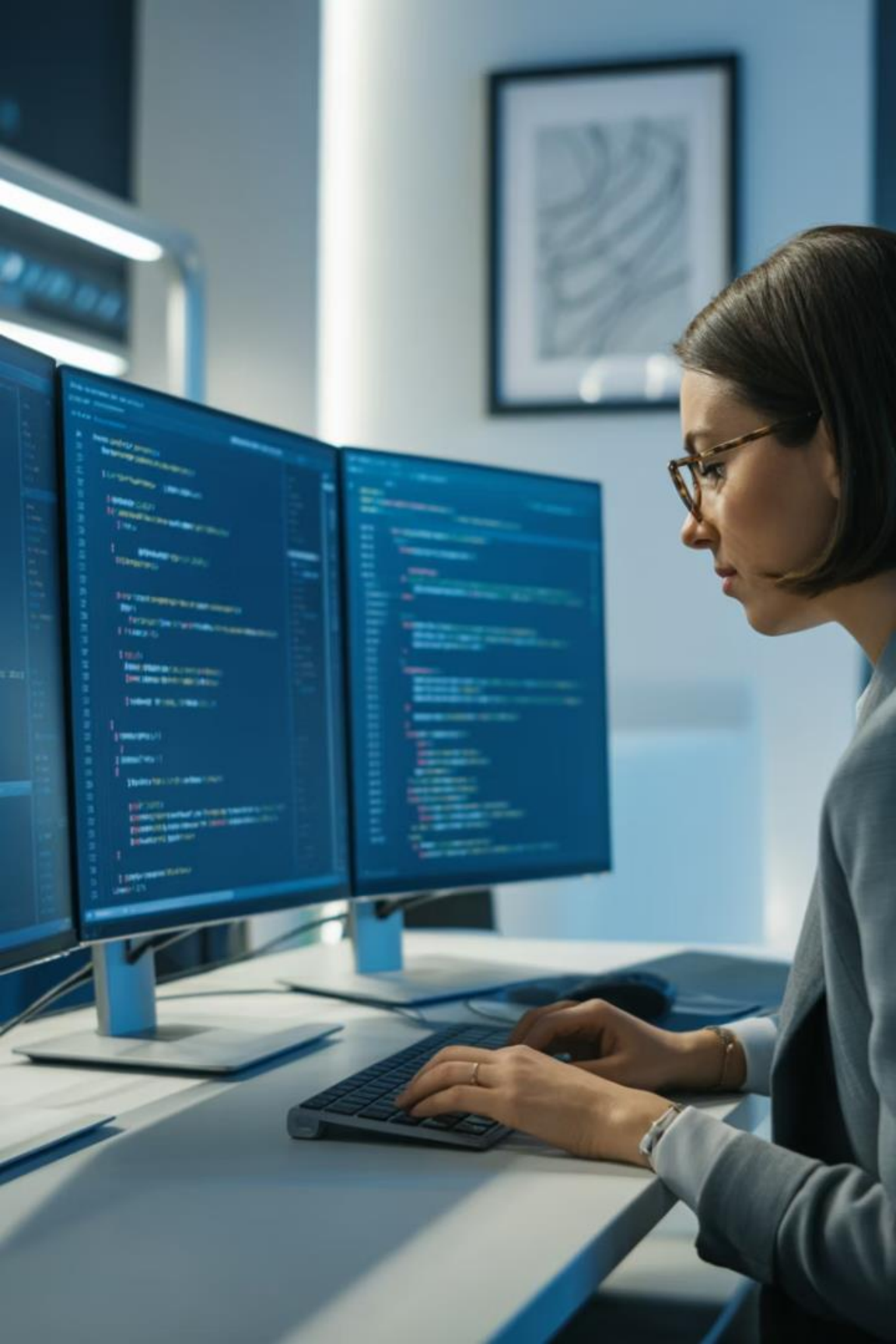
Moderný, rýchly a výkonný framework pre vývoj webových API v Pythone. Kombinuje vysokú výkonnosť s jednoduchou syntaxou a skvelou vývojárskou skúsenosťou.



Miroslav Reiter

fastapi





Čo je FastAPI?



Vysoká výkonnosť

Jeden z najrýchlejších Python webových frameworkov dostupných na trhu.

Moderná syntax

Využíva najnovšie funkcie Pythonu 3.7+ pre čistý a efektívny kód.



Bezpečnosť a validácia

Automatická validácia vstupov a typová kontrola ako v silno typovaných jazykoch.

Pozícia, kľúčové slovo alebo firma

fastapi x pridajte ďalšie...

Obec, okres, kraj alebo krajina

Skúste napríklad Bratislava

+30km

Vyhľadať

Rozšírené vyhľadávanie

KRAJE	
Bratislavský kraj	233
Košický kraj	98
Žilinský kraj	90
Trnavský kraj	90
Banskobystrický kraj	89
Nitriansky kraj	82
Trenčiansky kraj	81
Prešovský kraj	58
» Zoznam lokalít	
» Zahraničie	

PRÁCA Z DOMU	
Práca iba na pracovisku	524
Práca iba z domu	39
Možná čiastočne	149

PLAT	
Vyberte obdobie	mesiac hodina
od 600 EUR	756
od 800 EUR	754
od 1 000 EUR	713
od 1 300 EUR	588
od 1 500 EUR	498
od 1 800 EUR	383
od 2 000 EUR	354
od 2 500 EUR	240

POZÍCIA	
Programátor	176
UX dizajnér	65

PONUKY PRÁCE: fastapi

1 - 20

Junior Django (Python) programátor

AnimalSoft, s. r. o.

Práca z domu

1 700 - 2 200 EUR/mesiac

Можливість для людей з України

Reagujte bez životopisu

Uložiť ponuku

Pred mesiacom

Medior Python Developer

GymBeam s.r.o.

Košice, Slovensko (Pozícia umožňuje občasnú prácu z domu)

2 000 - 3 300 EUR/mesiac

Uložiť ponuku

Pred 5 dňami

Flutter Developer (Android)

DefTech a.s.

Remote work

2 000 - 4 200 EUR/mesiac

Uložiť ponuku

Pred 2 týždňami

Získaj skúsenosti s ERP - Python ERP Developer - Študent SŠ, VŠ - až 1600€ v čistom

4B2B consulting s. r. o.

Práca z domu

Ponuka onedlho končí!

1 600 EUR/mesiac

Uložiť ponuku

Pred 2 týždňami

Python Developer/ka

Websupport, s.r.o.

Bratislava, Slovensko (Pozícia umožňuje občasnú prácu z domu)

2 500 EUR/mesiac

Uložiť ponuku

Pred týždňom

Python Developer – Přepis Aplikací (Brno)

radixal s.r.o.

Brno, Česko (Pozice umožňuje občasnou práci z domu)

40 000 - 120 000 Kč/mesiac

Python Developer/ka

Websupport, s.r.o.



Miesto práce

Bratislava, Slovensko (Pozícia umožňuje občasnú prácu z domu)



Druh pracovného pomeru

plný úväzok



Mzdové podmienky (brutto)

2 500 EUR/mesiac

Informácie o pracovnom mieste

Náplň práce, právomoci a zodpovednosti

Na čo sa u nás môžeš tešiť?

- Budeš pracovať na back-end systémoch, pomocou ktorých ovládame množstvo prvkov našej infraštruktúry. Prostredníctvom Python kódu budeš transformovať požiadavky z front-end aplikácií do akcií vykonávaných priamo na webových, mailových alebo databázových serveroch. Stretneš sa s technológiami ako MySQL, MongoDB, RabbitMQ, Redis, ZFS, Apache, Nginx a ďalšími.
- Každý aspekt našej infraštruktúry je ovládaný cez HTTP API, takže sa ti zídu skúsenosti s webovým frameworkom (Flask, **FastAPI**...) a ORM (používame SQLAlchemy nad MariaDB). Okrem API apiek sa však treba starať aj o kopec adminských skriptov a daemonov, prípadne o RPC funkcie ktoré na "druhom konci" vykonávajú príkazy z API aplikácií.
- Budeš súčasťou menšieho, stabilného, 7-8 členného tímu, ktorý sa skladá najmä z bývalých adminov Čas trávime buď na väčších projektoch v spolupráci s PHP a Admin/Engineering tímom, alebo na interných požiadavkách iných oddelení a operatívne vlastných systémov (zlepšenia + opravy bugov). Snažíme sa pokryť čo najviac kódu automatizovanými unit testami, alebo integračnými testami a na manuálne testovanie používame lokálne prostredie alebo vzdialené "staging" prostredie.

Očakávame od teba



Reagovať na ponuku



Odporučiť ponuku známemu



Odporučiť ponuku cez Facebook



Tlačiť



Uložiť ponuku



Nahlásiť pracovnú ponuku



Ďalšia pracovná ponuka v zozname

Inštalácia FastAPI

Príkaz	Obsahuje	Vhodné pre
<code>fastapi</code>	iba základné API funkcie	pokročilí používatelia
<code>fastapi[standard]</code>	FastAPI + Uvicorn + doplnky	začiatočníci a vývojári

New Project

 Pure Python

Python

 Django

 FastAPI

 Flask

 Pyramid

 Jupyter

 dbt

> Other

Location: C:\Users\miros\PycharmProjects\FastAPIProject

☒ Create Git repository

Interpreter type: Project venv Base conda Custom environment

Python version:  C:\Program Files\Python312\python.exe detected in the system

Python virtual environment will be created in the project root:
C:\Users\miros\PycharmProjects\FastAPIProject\.venv

Create

Cancel

FA

FastAPIProject

main

FastAPIProject

main.py

test_main.http

Project

FastAPIProject

.venv

library root

Lib

Scripts

.gitignore

pyvenv.cfg

main.py

test_main.http

External Libraries

Scratches and Consoles

1

from fastapi import FastAPI

2

3

app = FastAPI()

4

5

6

@app.get("/")

7

async def root():

8

return {"message": "Hello World"}

9

10

11

@app.get("/hello/{name}")

12

async def say_hello(name: str):

13

return {"message": f"Hello {name}"}

14

Run

FastAPIProject

C:\Users\miros\PycharmProjects\FastAPIProject\.venv\Scripts\python.exe -m uvicorn main:app --reload

INFO: Will watch for changes in these directories: ['C:\\Users\\miros\\PycharmProjects\\FastAPIProject']

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

INFO: Started reloader process [17620] using StatReload

INFO: Started server process [39708]

INFO: Waiting for application startup.

INFO: Application startup complete.

INFO: 127.0.0.1:64065 - "GET / HTTP/1.1" 200 OK

INFO: 127.0.0.1:64065 - "GET /favicon.ico HTTP/1.1" 404 Not Found

FastAPIProject

main.py

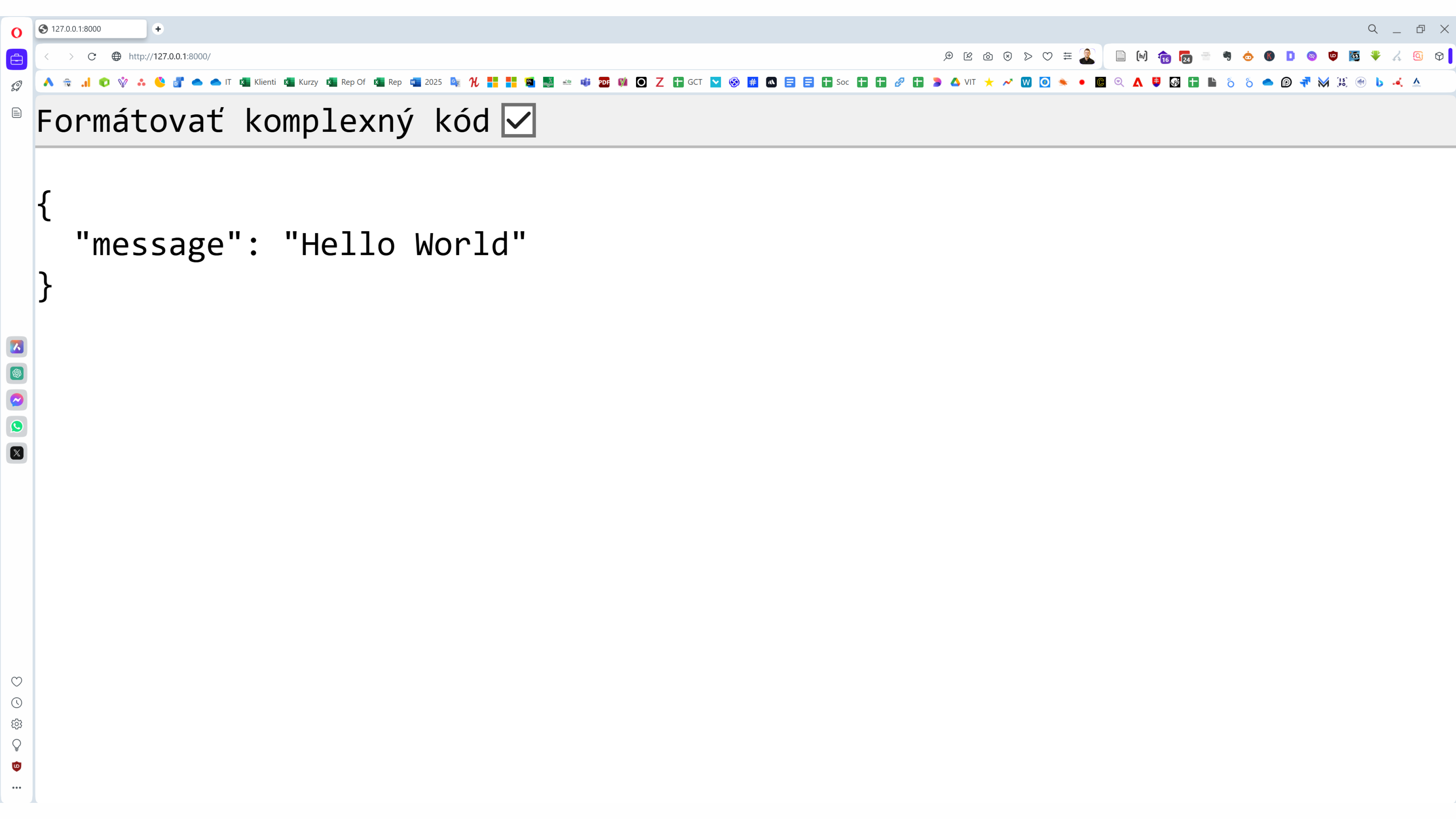
12:32

CRLF

UTF-8

4 spaces

Python 3.12 (FastAPIProject)



default

GET	/ Root	
GET	/hello/{name} Say Hello	

Schemas

- HTTPValidationError > Expand all object
- ValidationError > Expand all object

POST

/items/ Create Item

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value

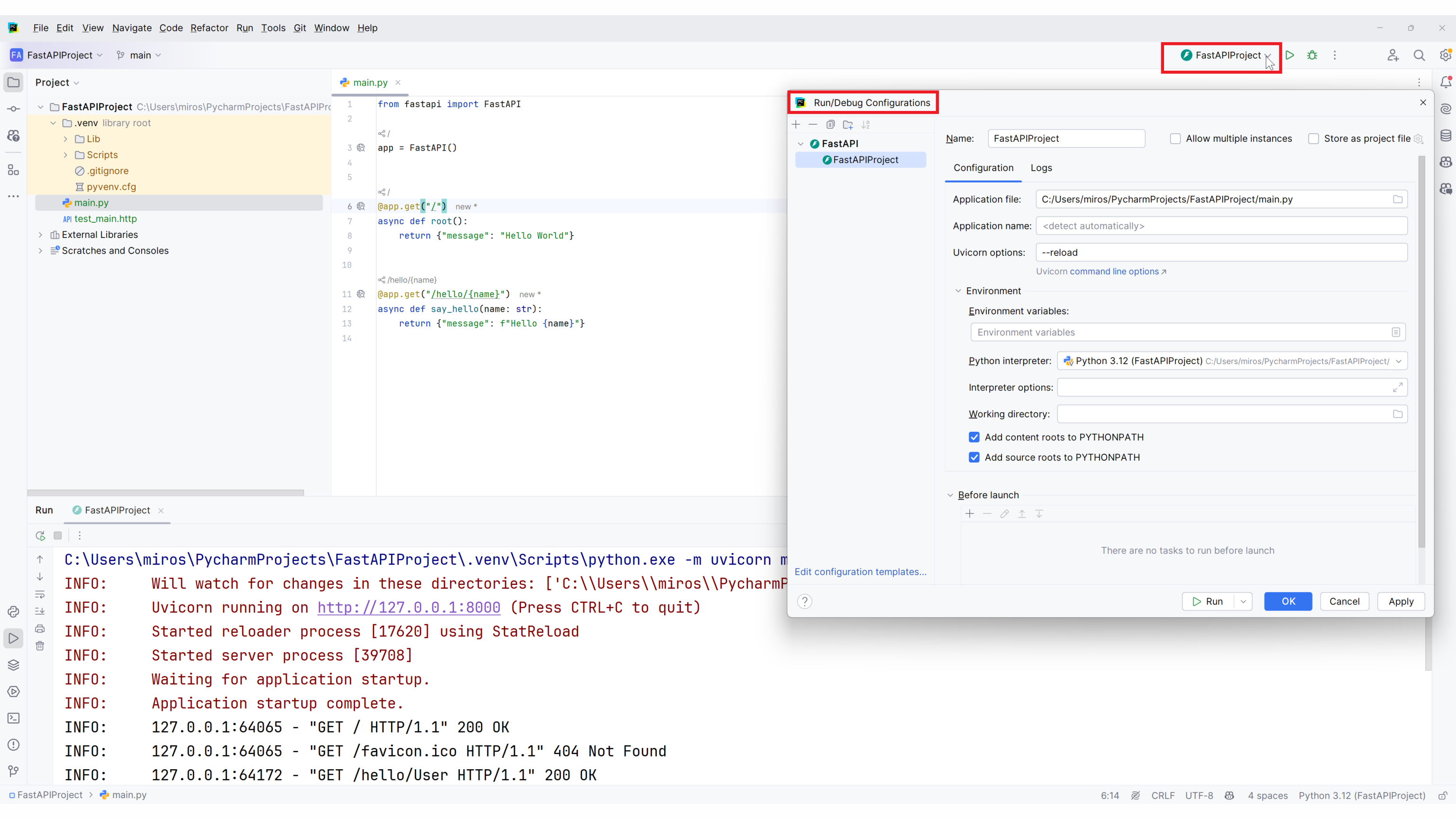
Schema

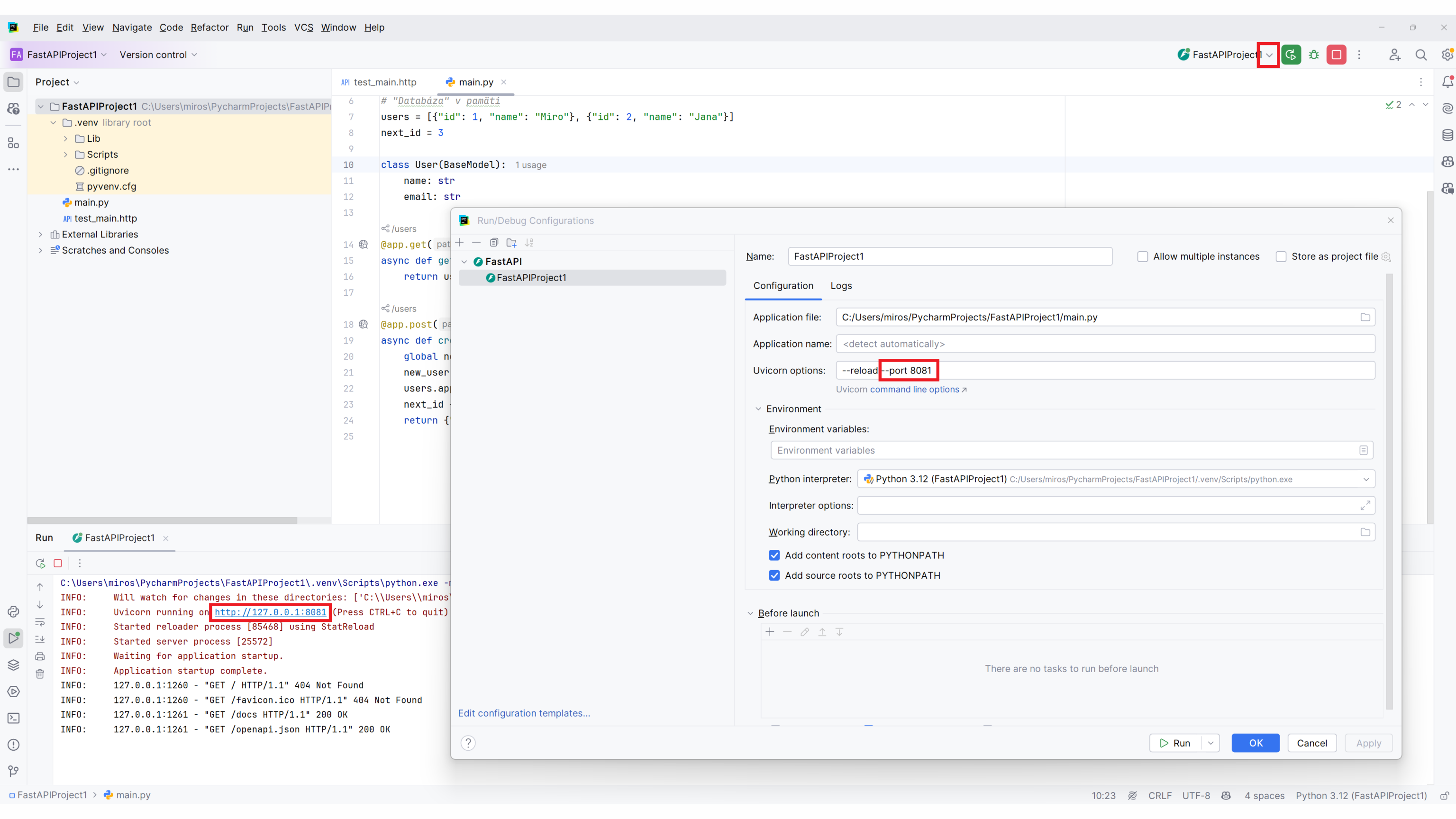
```
{  "name": "string",  "price": 0}
```

Responses

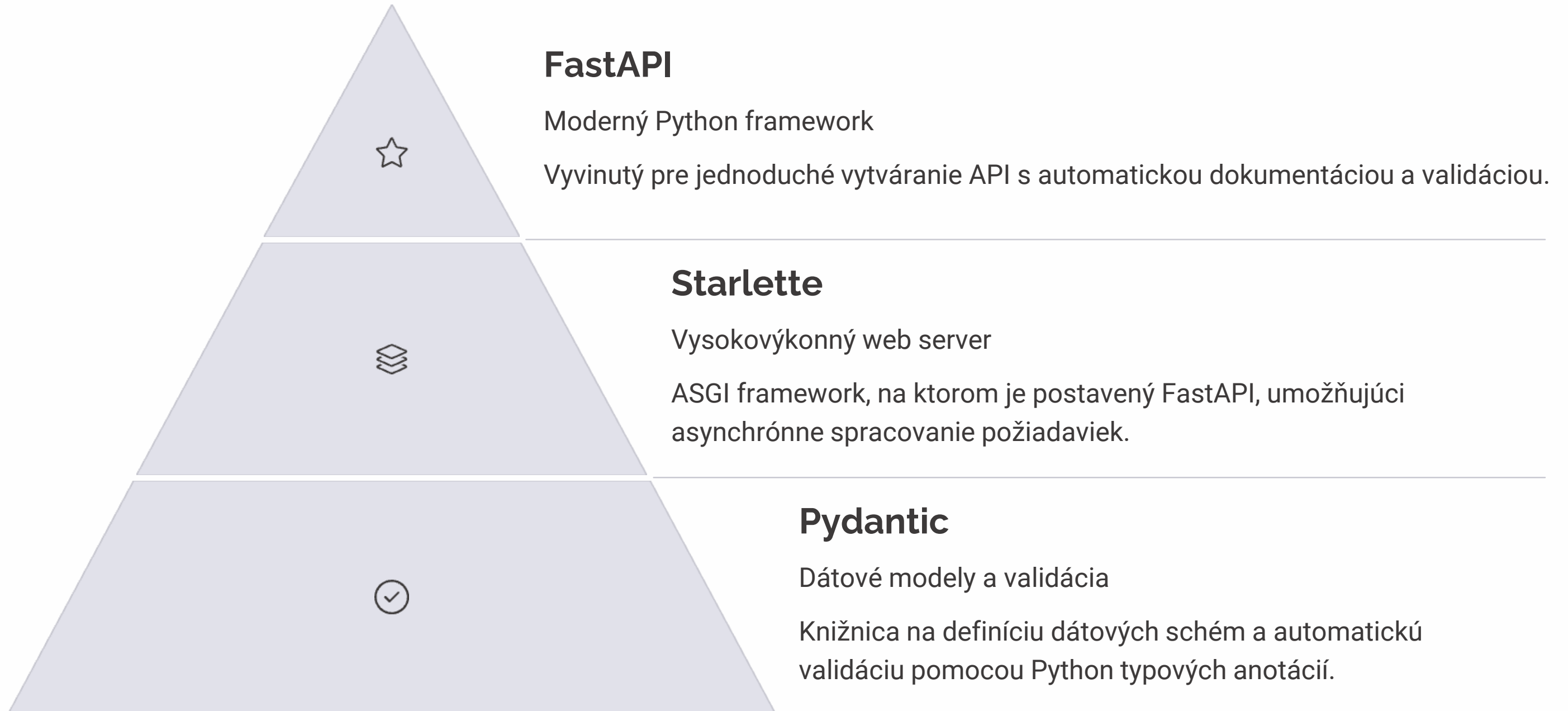
Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }]}</pre>	No links

Schemas





Technologické základy





Ideálne použitie

Mikroslužby

Rýchle a efektívne služby pre distribuované systémy.

Poznámka: Umožňuje jednoduché škálovanie a údržbu jednotlivých komponentov bez ovplyvnenia celého systému.

REST API

Tvorba robustných API endpointov s automatickou dokumentáciou.

Poznámka: Integrovaný Swagger/OpenAPI poskytuje interaktívnu dokumentáciu bez dodatočnej práce.

Backend pre frontend

Spoločné backend riešenia pre moderné frontend aplikácie.

Poznámka: Asynchrónne spracovanie požiadaviek zaisťuje optimálny výkon aj pri vysokej záťaži.

ML služby

Nasadenie modelov strojového učenia ako API služieb.

Poznámka: Jednoduchá integrácia s populárnymi ML knižnicami ako TensorFlow, PyTorch a scikit-learn.

Hlavné výhody



Vysoká výkonnosť

Porovnateľná s Node.js a Go, oveľa rýchlejšia než Flask či Django.

Poznámka: Stavia na asyncio a uvicorn ASGI server, čo umožňuje spracovať tisíce požiadaviek za sekundu.



Skvelá DX

Intuitívne API a vynikajúca dokumentácia pre vývojárov.

Poznámka: Automaticky generovaná interaktívna dokumentácia pomocou Swagger UI a ReDoc uľahčuje testovanie a používanie API.



Moderná syntax

Využíva Python 3.7+ funkcie pre čistý a efektívny kód.

Poznámka: Podporuje typové anotácie, asynchrónne funkcie a ďalšie moderné Python prvky pre lepšiu čitateľnosť kódu.



Bezpečnosť

Automatická validácia a typová kontrola pre bezpečné API.

Poznámka: Integrovaná validácia pomocou Pydantic zabezpečuje, že všetky dáta sú správne validované a typované, čím sa predchádza bezpečnostným problémom.

Nastavenie prostredia

Inštalácia

Nainštalujte FastAPI a Uvicorn pomocou pip:

```
pip install fastapi uvicorn
```

Poznámka: Uvicorn je ASGI server potrebný na spustenie FastAPI aplikácií. Môžete tiež nainštalovať voliteľné závislosti pomocou `pip install fastapi[all]`.

Vytvorenie projektu

Vytvorte súbor `main.py` s vašou aplikáciou.

Poznámka: V tomto súbore budete definovať všetky API endpointy. Začnite vytvorením základnej aplikácie pomocou `app = FastAPI()`.

Spustenie servera

Spustite vývojový server pomocou:

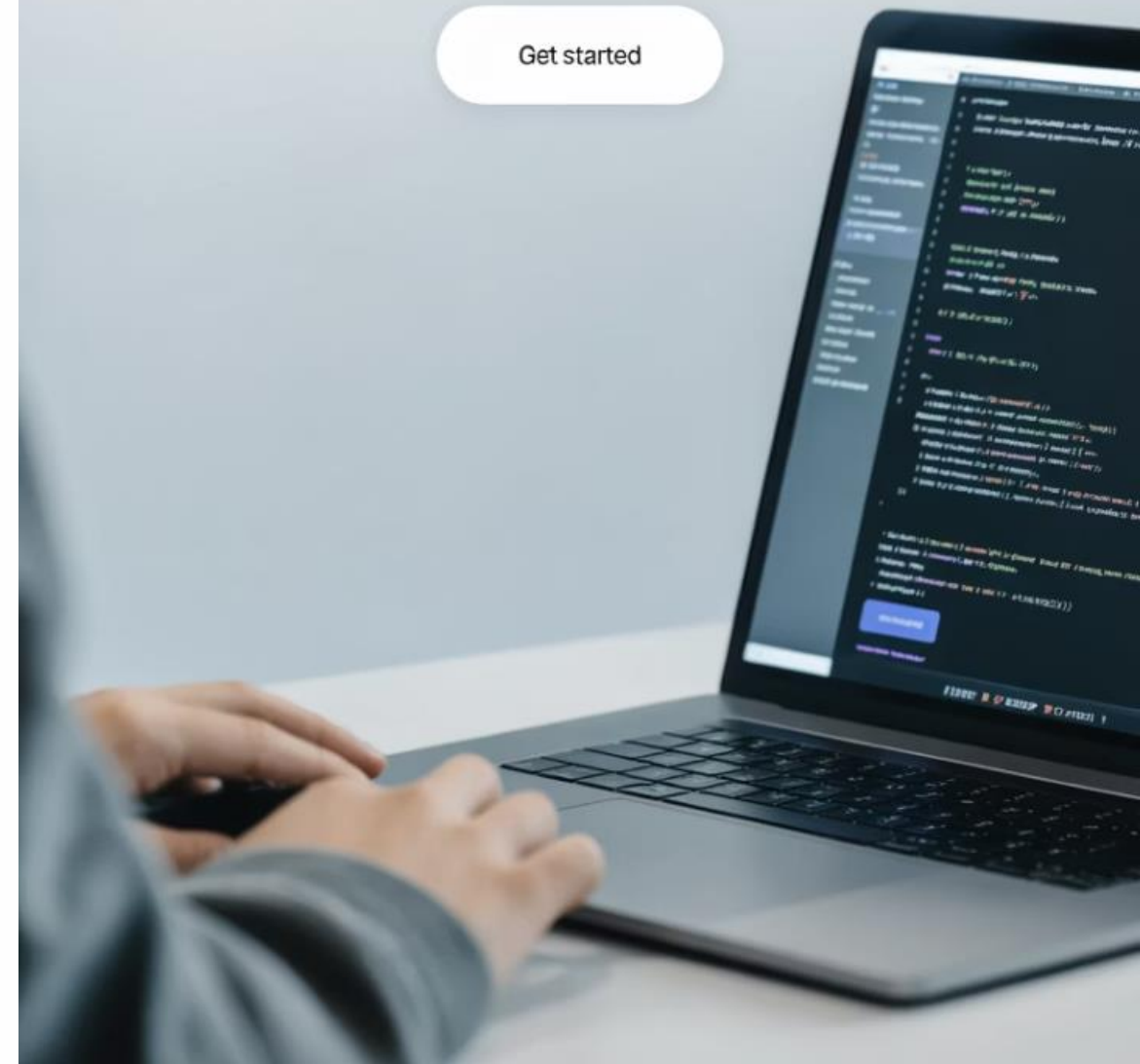
```
uvicorn main:app --reload
```

Poznámka: Parameter `--reload` zabezpečí automatické reštartovanie servera pri zmenách v kóde, čo je užitočné počas vývoja. Pre produkčné nasadenie by ste mali zvážiť konfiguráciu s viacerými pracovnými vláknami.

Streamline Your Workflow

Accelerate development with our Ai-powered tools

Get started





Voliteľné závislosti



python-multipart

Pre spracovanie formulárových dát a súborov.



jinja2

Šablónovací systém pre HTML stránky.



aiofiles

Asynchrónna práca so súbormi.

Prvý endpoint



Import knižnice

```
from fastapi import FastAPI
```

Importujeme základnú triedu FastAPI, ktorá je jadrom celého frameworku a umožňuje vytvárať API endpointy.



Vytvorenie aplikácie

```
app = FastAPI()
```

Vytvoríme inštanciu FastAPI aplikácie, ktorá bude slúžiť ako hlavný objekt pre definovanie endpointov a nastavení servera.



Definovanie endpointu

```
@app.get("/") def read_root(): return {"message": "Hello, FastAPI"}
```

Pomocou dekorátora `@app.get()` definujeme endpoint pre HTTP metódu GET na ceste `/` (koreňová cesta), ktorý vráti JSON s uvítacou správou.

Tieto tri kroky sú minimálne požiadavky na vytvorenie funkčného FastAPI endpointu, ktorý dokáže prijímať HTTP požiadavky a odpovedať na ne.

Testovanie API

Swagger UI

Interaktívna dokumentácia dostupná na:

<http://127.0.0.1:8000/docs>

Umožňuje testovať všetky endpointy priamo z prehliadača.

Poznámka: Swagger UI generuje dokumentáciu automaticky na základe vašich endpointov a ich popisov. Výborne sa hodí pre vývojárov, ktorí potrebujú rýchlo otestovať funkcionality.

ReDoc

Alternatívna dokumentácia dostupná na:

<http://127.0.0.1:8000/redoc>

Prehľadné zobrazenie API dokumentácie v štýle referenčnej príručky.

Poznámka: ReDoc poskytuje prehľadnejšie čítanie pre koncových používateľov API. Je optimalizovaný pre mobilné zariadenia a podporuje rozšírené OpenAPI špecifikácie.

Úlohy FastAPI

1. Čo je to API, FastAPI, Pydantic?
2. Aké sú kľúčové komponenty?
3. Aké sú hlavné vlastnosti?
4. Čo si treba nainštalovať?
5. Ako začať používať FastAPI
6. Ako sa pracuje s portami?



FastAPI

Ako na Testovanie Požiadaviek 2

AKREDITOVANÝ KURZ



FastAPI





Čo sa naučíme?

1. Ako vytvoriť a testovať aplikáciu FastAPI Python + HTML + JS?
2. Čo sú to HTTP metódy, status kódy, endpoint routing?
3. Ako sa používajú GET, POST...?
4. Ako sa pracujeme s chybami?
5. Ako sa pracuje s pydantic?
6. Ako sa nastavuje PyCharm pre FastAPI?
7. Profit

Seooh

HOMEAPI REFERENCETUTORIALS

Login



Understanding the foundation of API communication

Request headers are the first part of an API request. They contain information about the client, the request, and the response. Headers are used to identify the client, the request, and the response. They are used to identify the client, the request, and the response. They are used to identify the client, the request, and the response.

Copyright © 2020 Seooh. All rights reserved. Terms of Service Privacy Policy

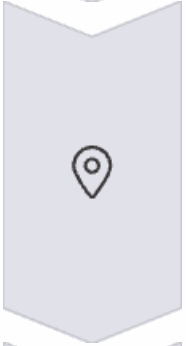
Spracovanie požiadaviek



Query parametre

Parametre v URL za otáznikom: ?q=hľadanie

Používajú sa na filtrovanie, radenie a stránkovanie výsledkov. Nemali by obsahovať citlivé údaje, keďže sú viditeľné v URL.



Path parametre

Parametre ako súčasť cesty: /pouzivatel/123

Identifikujú konkrétny zdroj alebo entitu. Sú povinné a priamo ovplyvňujú smerovanie požiadavky na správny endpoint.



Request body

Dáta posielané v tele požiadavky (JSON, formuláre)

Vhodné pre komplexné dáta a väčšie objemy informácií. Používa sa najmä pri POST, PUT a PATCH metódach na vytváranie alebo aktualizáciu zdrojov.



Headers

Hlavičky požiadavky, napr. Authorization

Obsahujú metadáta o požiadavke ako autentifikačné tokeny, informácie o formáte obsahu (Content-Type) alebo preferovanom jazyku (Accept-Language).

⚡ API Quick Reference ⚡

💠 HTTP Verbs

- └ 🚀 **GET** : Retrieve data from the server
- └ 🚀 **POST** : Send data to the server to create a resource
- └ 🚀 **PUT** : Send data to the server to update a resource
- └ 🚀 **PATCH** : Send data to the server to update a resource partially
- └ 🚀 **DELETE** : Delete a resource from the server
- └ 🚀 **TRACE** : Returns the full HTTP request received by the server for debugging and diagnostic purposes
- └ 🚀 **OPTIONS** : Returns the HTTP methods supported by the server for the requested URL
- └ 🚀 **CONNECT** : Converts the request connection to a transparent TCP/IP tunnel for secure communication
- └ 🚀 **PURGE** : Invalidates a cached resource
- └ 🚀 **LOCK** : Locks the resource for exclusive use by the client
- └ 🚀 **UNLOCK** : Unlocks the resource previously locked by the client
- └ 🚀 **MKCOL** : Creates a new collection resource
- └ 🚀 **COPY** : Copies the resource identified by the Request-URI to the destination URI.

💠 HTTP Status Codes

- └ 🟡 **1xx** : Informational
- └ 🟢 **2xx** : Success
- └ 🟡 **3xx** : Redirection
- └ 🟡 **4xx** : Client Errors
- └ 🟡 **5xx** : Server Errors

⚡ API Quick Reference ⚡

💠 Response Headers

- └ 🌐 **Content-Type** : Specifies the MIME type of the data in the response body
- └ 🌐 **Content-Length** : Specifies the length of the response body in bytes
- └ 🌐 **Cache-Control** : Specifies the caching behavior of the response
- └ 🌐 **Location** : Specifies the URI of a resource that can be used to retrieve the requested resource
- └ 🌐 **Server** : Specifies the name and version of the server software that generated the response
- └ 🌐 **Access-Control-Allow-Origin** : Specifies which origins are allowed to access the resource
- └ 🌐 **Set-Cookie** : Specifies a cookie that should be stored by the client and sent back to the server with future requests
- └ 🌐 **Expires** : Specifies the date and time after which the response is considered stale
- └ 🌐 **Last-Modified** : Specifies the date and time the resource was last modified.

💠 API Design

- └ 🏠 **REST** : Representational State Transfer, a design pattern for building web services
- └ 🏠 **SOAP** : Simple Object Access Protocol, a messaging protocol for exchanging structured data
- └ 🏠 **GraphQL** : A query language and runtime for building APIs
- └ 🏠 **API Gateway** : A service that manages, protects, and scales APIs

⚡ API Quick Reference ⚡

💠 API Architectures

- └  **SOA** : Service-Oriented Architecture, an architectural style for building distributed systems
- └  **Microservices** : An architectural style for building complex applications as a suite of small, independent services
- └  **Serverless** : A cloud computing execution model where the cloud provider manages the infrastructure and automatically allocates resources as needed
- └  **Event-Driven** : An architectural style where the flow of data between components is triggered by events
- └  **RESTful API** : An architectural style that uses HTTP requests to GET, POST, PUT, and DELETE data.

💠 API Design Patterns

- └  **Adapter Pattern** : A pattern that converts the interface of a class into another interface that clients expect
- └  **Decorator Pattern** : A pattern that adds behavior to an individual object dynamically
- └  **Proxy Pattern** : A pattern that provides a surrogate or placeholder for another object to control access to it
- └  **Chain of Responsibility Pattern** : A pattern that delegates commands to a chain of processing objects
- └  **Observer Pattern** : A pattern that defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

⚡ API Quick Reference ⚡

💠 API Security

- └  **OAuth** : An open standard for authorization used for protecting APIs
- └  **JWT** : JSON Web Tokens, a standard for securely transmitting information between parties as a JSON object
- └  **SSL/TLS** : Secure Sockets Layer/Transport Layer Security, a protocol for establishing a secure connection between a client and a server
- └  **API Key** : A secret token used to authenticate API requests
- └  **Rate Limiting** : A technique used to limit the number of requests that can be made to an API over a specific period of time
- └  **OpenID Connect** : An authentication layer built on top of OAuth that allows users to be authenticated across multiple domains
- └  **Cross-Origin Resource Sharing (CORS)** : A mechanism that allows many resources (e.g., fonts, JavaScript, etc.) on a web page to be requested from another domain outside the domain from which the resource originated

💠 API Testing

- └  **Postman** : A popular tool for testing and debugging APIs
- └  **SoapUI** : A tool for testing SOAP and REST web services
- └  **Swagger** : A tool for designing, building, and testing APIs
- └  **JMeter** : A tool for testing the performance of APIs
- └  **TestRail** : A test management tool for planning, executing, and tracking API tests
- └  **Dredd** : A command-line tool for testing API documentation against its backend implementation
- └  **REST Assured** : A Java-based library for testing RESTful APIs

Prvý endpoint



Import knižnice

```
from fastapi import FastAPI
```

Importujeme základnú triedu FastAPI, ktorá je jadrom celého frameworku a umožňuje vytvárať API endpointy.



Vytvorenie aplikácie

```
app = FastAPI()
```

Vytvoríme inštanciu FastAPI aplikácie, ktorá bude slúžiť ako hlavný objekt pre definovanie endpointov a nastavení servera.



Definovanie endpointu

```
@app.get("/") def read_root(): return {"message": "Hello, FastAPI"}
```

Pomocou dekorátora `@app.get()` definujeme endpoint pre HTTP metódu GET na ceste `/` (koreňová cesta), ktorý vráti JSON s uvítacou správou.

Tieto tri kroky sú minimálne požiadavky na vytvorenie funkčného FastAPI endpointu, ktorý dokáže prijímať HTTP požiadavky a odpovedať na ne.

Testovanie API

Swagger UI

Interaktívna dokumentácia dostupná na:

<http://127.0.0.1:8000/docs>

Umožňuje testovať všetky endpointy priamo z prehliadača.

Poznámka: Swagger UI generuje dokumentáciu automaticky na základe vašich endpointov a ich popisov. Výborne sa hodí pre vývojárov, ktorí potrebujú rýchlo otestovať funkcionality.

ReDoc

Alternatívna dokumentácia dostupná na:

<http://127.0.0.1:8000/redoc>

Prehľadné zobrazenie API dokumentácie v štýle referenčnej príručky.

Poznámka: ReDoc poskytuje prehľadnejšie čítanie pre koncových používateľov API. Je optimalizovaný pre mobilné zariadenia a podporuje rozšírené OpenAPI špecifikácie.

default



GET	/	Root	▼
GET	/favicon.ico	Favicon	▼
GET	/users	Get Users	▼
POST	/users	Create User	▼
PUT	/users/{user_id}	Update User	▼
PATCH	/users/{user_id}	Patch User	▼
DELETE	/users/{user_id}	Delete User	▼
POST	/products	Add Product	▼
GET	/crash	Crash	▼

Schemas



HTTPValidationError: FastAPI

FastAPI (0.1.0)

Download OpenAPI specification: [Download](#)

Get Users

Responses

✓ 200 Successful Response

RESPONSE SCHEMA: application/json

any

GET /users

http://127.0.0.1:8000/users

200

Content type
application/json

Copy

null



OPENDATA API



OpenData FS API

Vitajte na portáli OpenData FS API. Cieľom tejto stránky je sprístupniť vývojárom aplikácií údaje dostupné na našom portáli. Aplikačné programové rozhranie alebo API je rozhranie, ktoré počítačová aplikácia alebo systém môže použiť na prístup k súboru funkcií alebo programov tretej strany. Na portáli Open data FS sú cez rozhranie API dostupné okrem všetkých datasetov aj exporty z vybraných online informačných zoznamov, ktoré FS zverejňuje na portáli FS.

[OpenData API](#)[Generovanie API kľúča](#)[Dokumentácia k rozhraniu API](#)

Aplikačné programové rozhranie alebo API je rozhranie, ktoré počítačová aplikácia alebo systém môže použiť na prístup k súboru funkcií alebo programov tretej strany.

Zmyslom API je zabezpečenie komunikácie medzi dvoma aplikáciami, ktoré si navzájom vymieňajú dáta. Umožňuje využívať už naprogramované riešenia a integrovať ich do vlastných web stránok alebo softvéru.

Prosím, vyplňte požadované údaje:

[Generovať API Kľúč](#)

Authentifikacia API

- prostredníctvom [API kľúča](#)

Limity API

- štandardný limit: **1 000** žiadostí za hodinu
- ak Vám limit nevyhovuje, požiadajte o zvýšenie prostredníctvom menu KONTAKT

Dôležité dokumenty

- [technická príručka - Opendata portál](#)
- [technická príručka - Informačné zoznamy na PFS](#)

Na portáli Open data FS sú cez rozhranie API dostupné okrem všetkých datasetov aj exporty z vybraných online informačných zoznamov, ktoré FS zverejňuje na portáli FS.

Ako používať OpenData FS API ?

API používa autorizačný kľúč pre prístup k api endpointom.

- 1. Registrujte sa, aby ste získali autorizačný kľúč.
- 2. Urobte GET požiadavku na ktorýkoľvek endpoint uvedený nižšie.
- 3. Vložte autorizačný kľúč do hlavičky GET požiadavky ako hodnotu "key".

napr.: `curl -H "key: KEY_VALUE" https://opendata.financnasprava.sk/api/dataset`

OpenData FS API 1.0.0 OAS3

[/doc/opendata.yaml](#)

API Documentation for OpenData FS API

Servers

`https://opendata.financnasprava.sk/api` ▾

Authorize 

Category ▾

GET	/api/category	List of categories	
GET	/api/category/{ id }	Category informations	
GET	/api/category/{ id }/datasets	List of datasets in category	

Dataset ▾

GET	/api/dataset	List of datasets	
GET	/api/dataset/{ id }	Dataset informations	

ITMS21+

Nové možnosti pre váš projekt

Zorientujte sa v aktuálnych možnostiach získania finančných prostriedkov z EÚ, ktorá v novom období pre Slovensko vyčlenila takmer 13 miliárd eur.



[Programy](#)



[Výzvy](#)



[Žiadosti](#)



[Projekty](#)

Ako predložiť žiadosť k výzve

Pozrite si jednoduchý postup, ako pracovať so systémom ITMS21+ a požiadajte o nenávratný finančný príspevok pre váš projekt.

[Pozrieť si postup](#)



Aktuality

[Technická odstavka systému ITMS21+ dňa 19.3.2025 vo večerných hodinách](#)

19. 3. 2025

ITMS21+ OpenData API, verzia: 1

2.6-RC4

OAS 3.0

/public/v1/openapi.json

REST API rozhranie systému ITMS21+

Servers

https://api.itms21.sk/public/v1

aktivitaprojekt

GET

/aktivitaprojekt

List objektov Aktivita projektu

GET

/aktivitaprojekt/id/{id}

Objekt Detail aktivity projektu

aktualita

GET

/aktualita/{uuid}

Objekt Aktualita

GET

/aktualita

List objektov Aktualita

ciselnik

GET

/ciselniky

List objektov Číselník

GET

/ciselniky/{ciselnikKod}/{id}

Objekt Hodnota číselníka

GET

/ciselniky/{ciselnikKod}

List objektov Hodnota číselníka

dodavatelobstaravatel

GET

/dodavatelobstaravatel/id/{id}

Objekt Dodavatel/Obstaravatel detail

dokument

GET

/dokument/{uuid}

Obsah objektu Dokument

program

GET

/financnyplanciele

List objektov Finančný plán ciele

ITMS 2014+ Open Data API, verzia: 2^{2.0}

/v2/swagger.json

REST API rozhranie systému ITMS2014+.
Update: vybrané zoznamy sú limitované na 1000000 záznamov.

Authorize



ciselnik

Informácie o číselníkoch



GET

/v2/ciselniky list ciselniky



GET

/v2/hodnotaCiselnika/{ciselnikKod} list hodnotaCiselnika



GET

/v2/hodnotaCiselnika/{ciselnikKod}/hodnota/{id} show hodnotaCiselnika



meta

Informácie o OpenData API



GET

/v2/meta/version version meta



nezrovnalost

Informácie o nezrovnalostiach



GET

/v2/nezrovnalost list nezrovnalost



GET

/v2/nezrovnalost/{id} show nezrovnalost



pohladavkovyDoklad

Informácie o pohľadávkových dokladoch



GET

/v2/pohladavkovyDoklad list pohladavkovyDoklad



GET

/v2/pohladavkovyDoklad/{id} show pohladavkovyDoklad



projekt

Informácie o projektoch



GET

/v2/aktivita list aktivitaProjektu



VŠEOBECNÉ INFORMÁCIE

REGISTER

ELEKTRONICKÉ SLUŽBY

Rýchle vyhľadávanie partnera

Vyhľadať podľa priezviska, obchodného mena alebo IČO



[Rozšírené vyhľadávanie](#)

[Kontakty](#)

[Vyhlásenie o prístupnosti](#)

[Odstávky](#)

[Nahlásiť problém](#)

RPVS Open Data API

REST API rozhranie systému RPVS

KonecniUzivateliovYhod

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

OpravneneOsoby

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

Partneri

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

PartneriVerejnehoSektora

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

VerejniFunkcionari

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

VerifikacneDokumenty

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

[BASE URL: /OpenData , API VERSION: v1]

[VALID](#) 

FAVICON GENERATOR

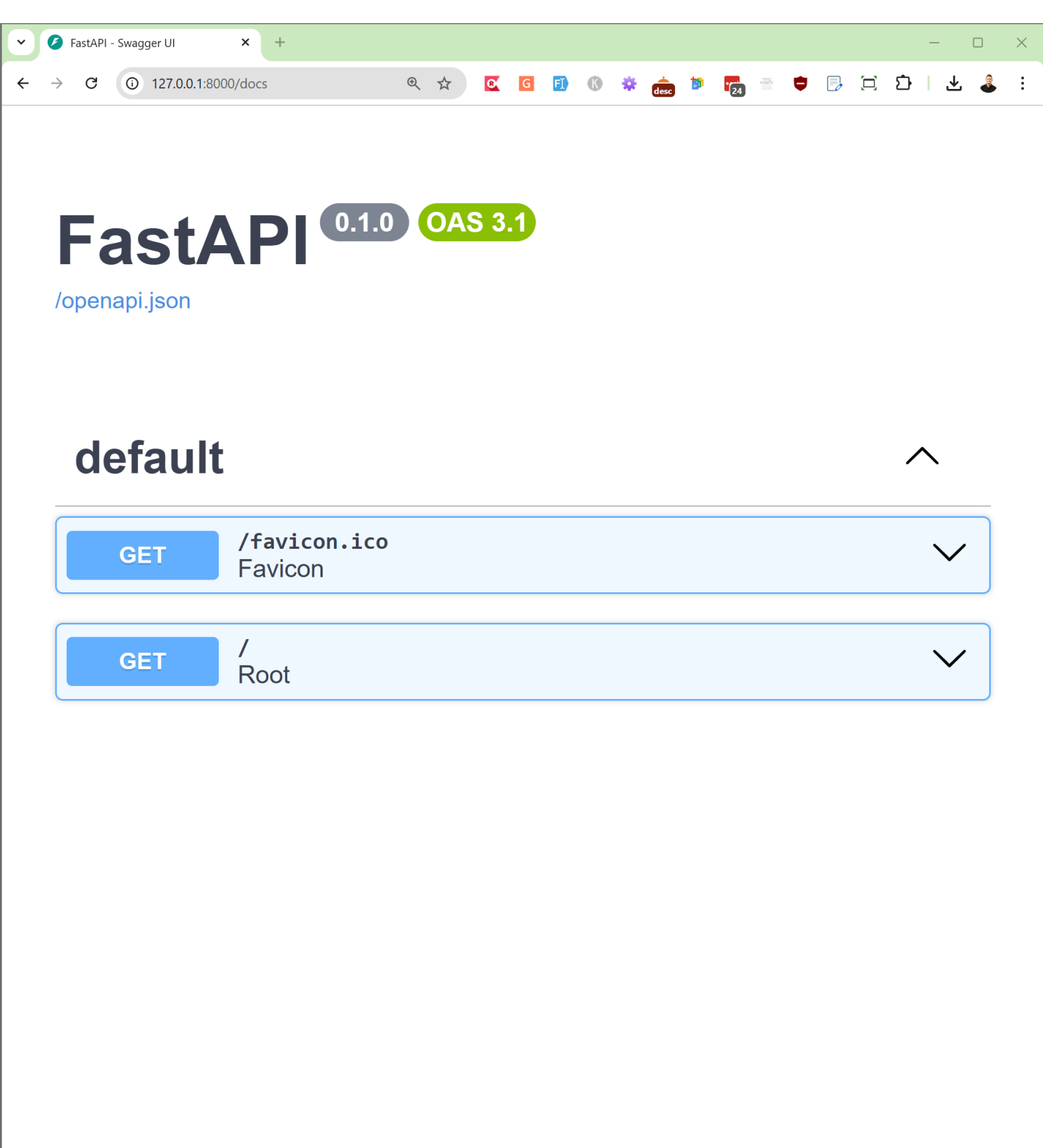
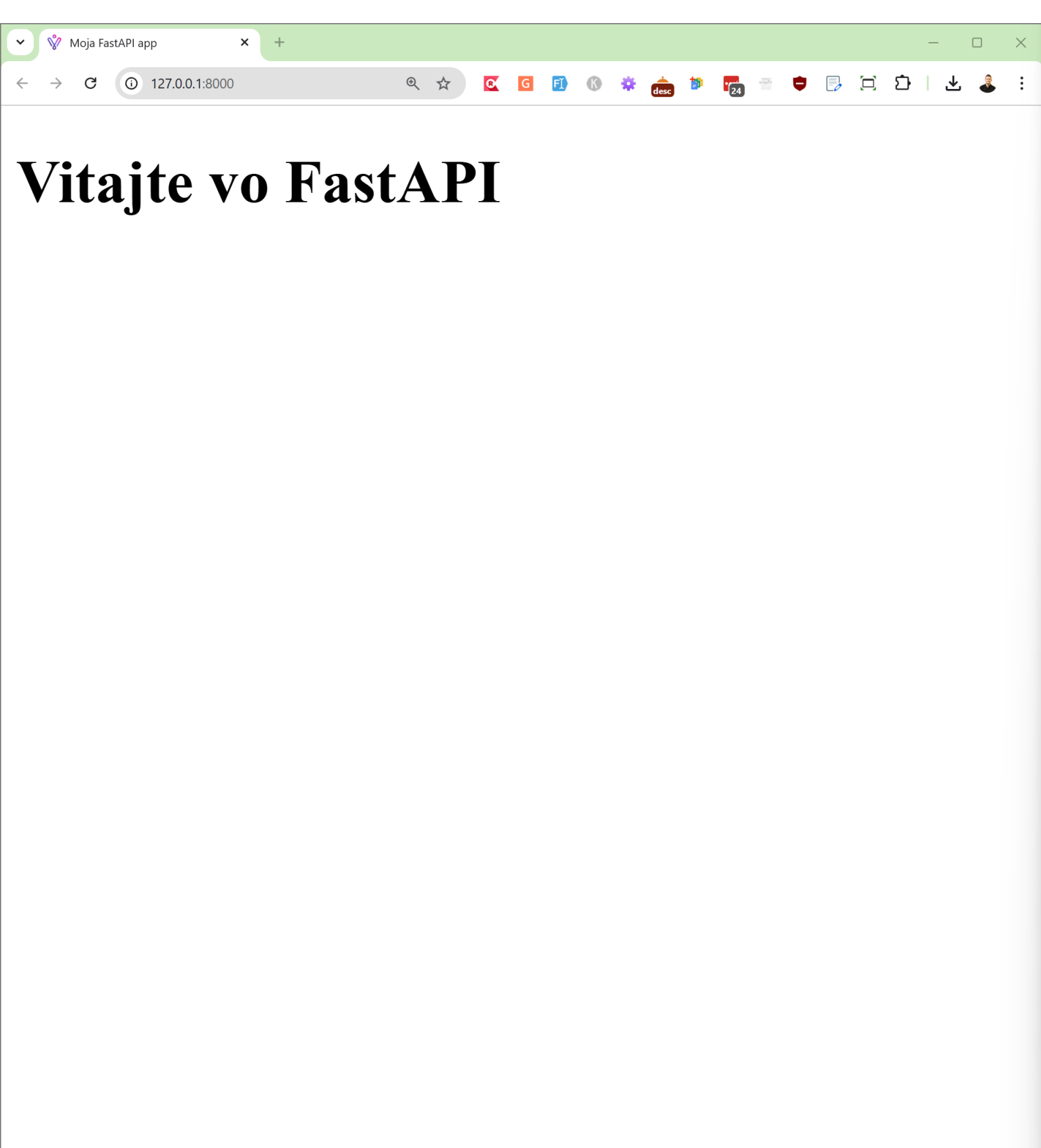
The only favicon generator you need for your next project. Quickly generate your favicon from text, image, or choose from hundreds of emojis.

Favicon Generators

PNG → ICO

TEXT → ICO

 **→ ICO**



default



GET	/	Root	▼
GET	/favicon.ico	Favicon	▼
GET	/users	Get Users	▼
POST	/users	Create User	▼
PUT	/users/{user_id}	Update User	▼
PATCH	/users/{user_id}	Patch User	▼
DELETE	/users/{user_id}	Delete User	▼
POST	/products	Add Product	▼
GET	/crash	Crash	▼

Schemas



HTTPValidationError: FastAPI

Registrácia používateľa

Meno



E-mail

Odoslať

Registrácia používateľa

Meno



E-mail

Odoslať

↓ Exportovať do CSV

Používateľ Adam bol zaregistrovaný

Zoznam používateľov

1: Miroslav – miroslav.reiter@gmail.com
2: Adam – adam@vita.sk

```
{
  "price": pat
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "price": pat
  }'
```

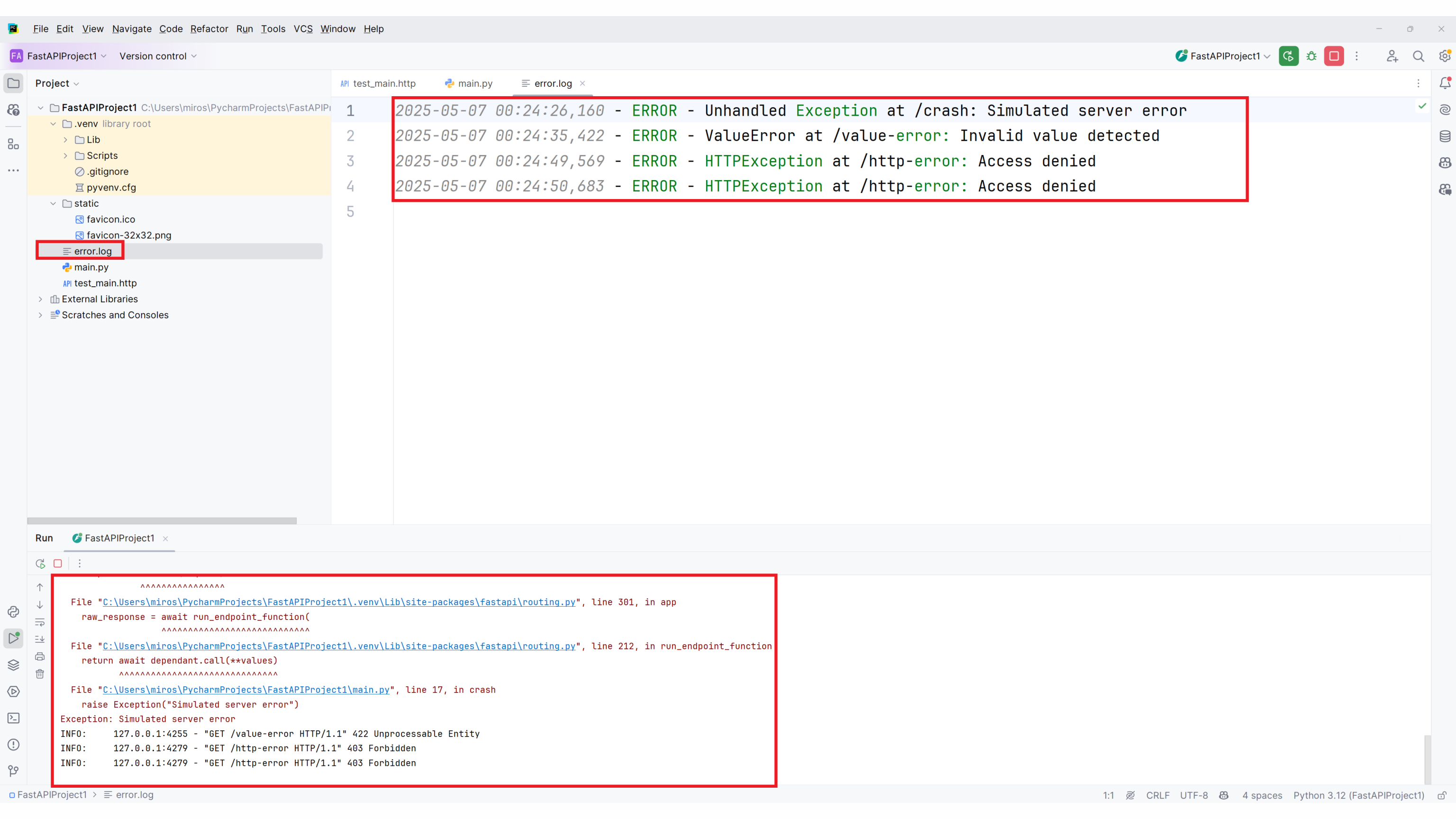
Request URL

```
http://127.0.0.1:8000/products
```

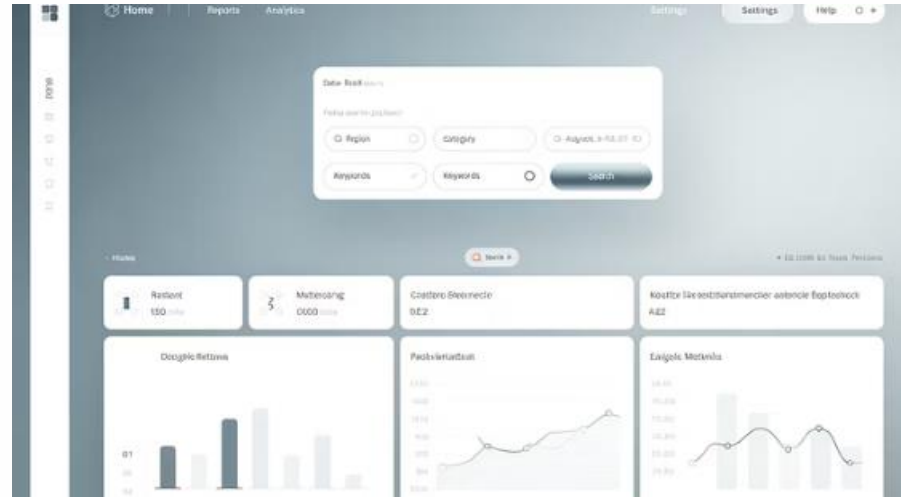
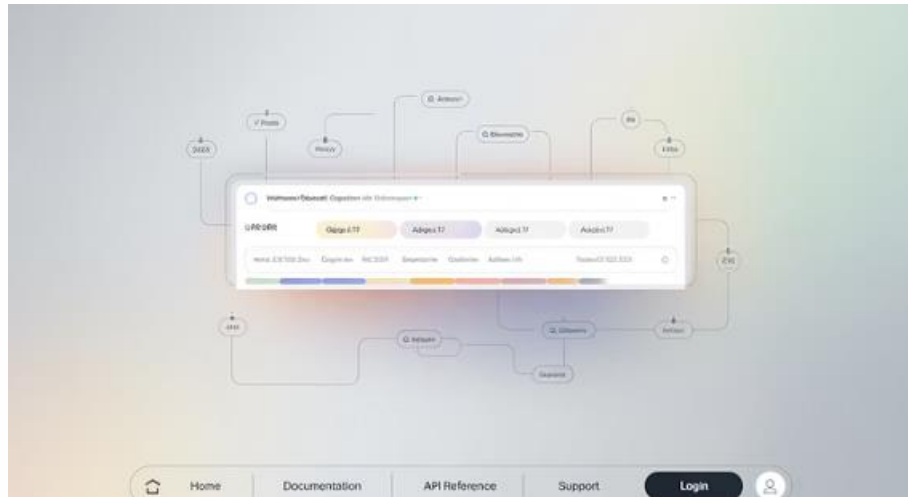
Server response

Code	Details
422	Error: Unprocessable Entity Response body

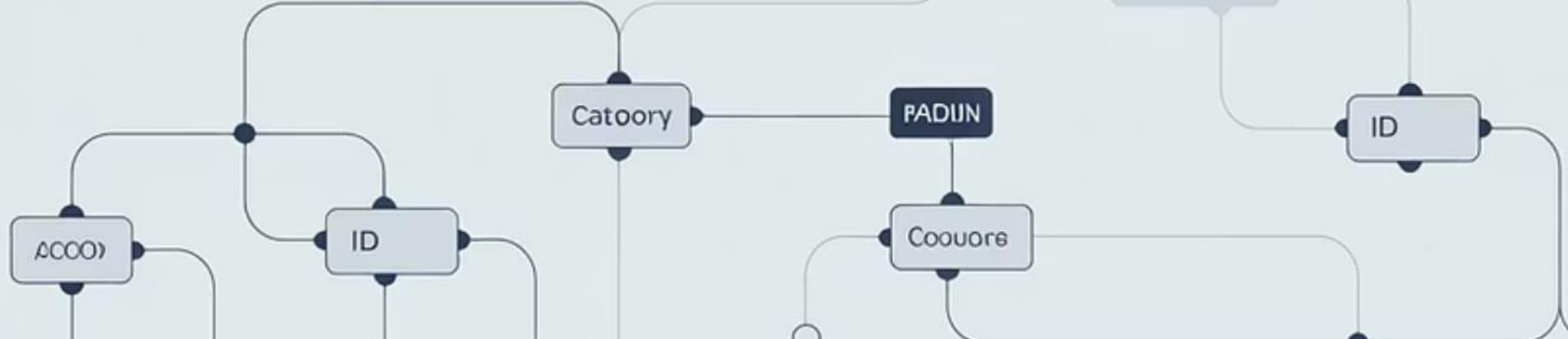
Internal Server Error



Query parametre



```
@app.get("/search")
def search(q: str = Query(default="", max_length=50)):
    return {"query": q}
```



Path parametre

Definícia cesty

```
@app.get("/pouzivatel/{pouzivatel_id}")
```

Poznámka: Cesta obsahuje dynamickú časť v zložených zátvorkách, ktorá sa mapuje na parameter funkcie

Spracovanie

```
return {"id": pouzivatel_id}
```

Poznámka: Hodnota parametra je použitá v odpovedi API, ktorá sa vráti klientovi ako JSON

1

2

3

Funkcia s parametrom

```
def info(pouzivatel_id: int):
```

Poznámka: Parameter 'pouzivatel_id' je automaticky extrahovaný z URL a konvertovaný na integer

Poznámka: Path parametre sú ideálne pre identifikátory a povinné hodnoty v API endpoints

Request body

Definícia modelu

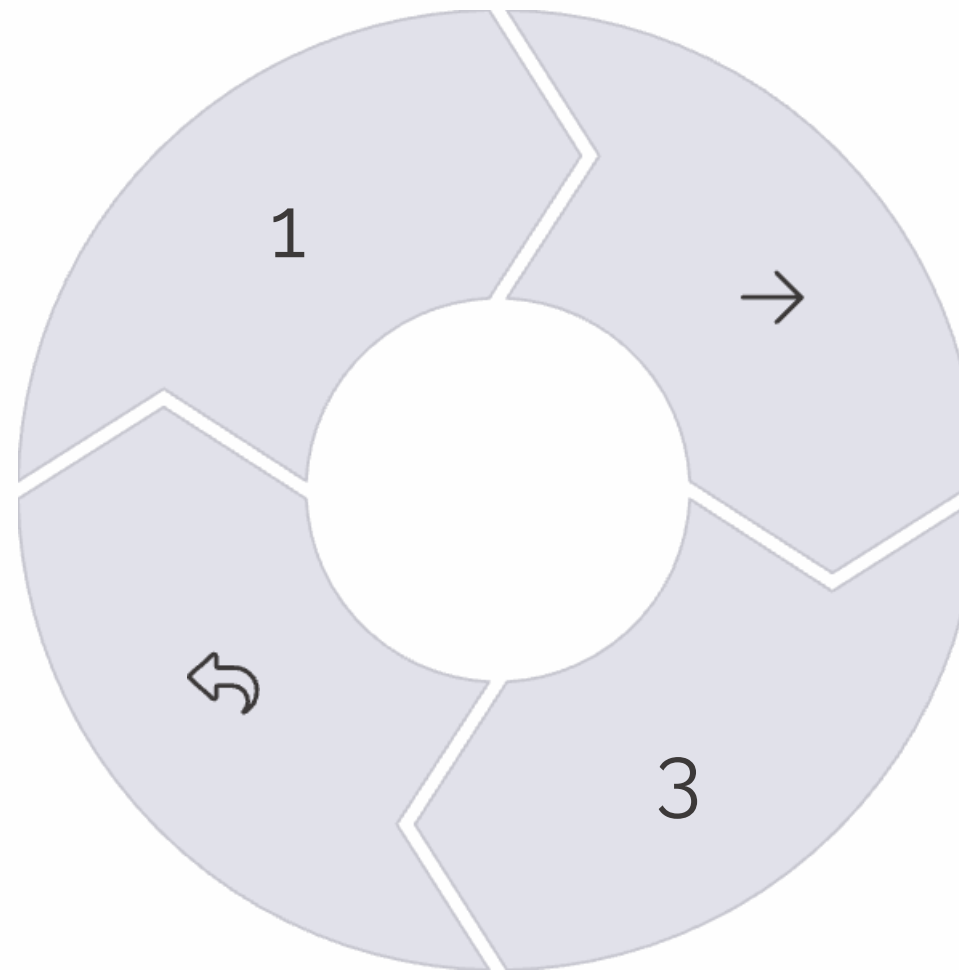
Vytvorenie Pydantic modelu pre validáciu dát

Trieda definuje štruktúru dát s anotáciou typov. Napr: `class User(BaseModel): name: str, age: int`

Odpoved'

Spracovanie a vrátenie výsledku

Po úspešnej validácii možno s dátami pracovať ako s objektom.
Možnosť vracať modely priamo ako odpoveď.



Prijatie dát

Použitie modelu ako parametra funkcie

Dáta z JSON požiadavky sa automaticky deserializujú. Napr: `@app.post("/users/")`
`def create_user(user: User):`

Validácia

Automatická kontrola typov a obmedzení

FastAPI kontroluje či JSON zodpovedá modelu. Neplatné dáta vracajú HTTP 422 s detailmi chýb.



Validácia pomocou Pydantic

100%

Typová kontrola

Všetky vstupy sú automaticky validované podľa typu.

0

Chyby typov

Eliminuje bežné chyby pri práci s dátami.

422

Status kód

Automatická odpoveď pri neplatných vstupoch.

Pydantic model

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
    email: str
    active: bool = True
```

Výhody

- Automatická validácia
- Dokumentácia v Swagger UI
- Predvolené hodnoty
- Konverzia typov
- IDE podpora (autocomplete)

Pokročilá validácia

Field validácia

Obmedzenia pre jednotlivé polia pomocou Field(...)

- min_length, max_length
- ge ($\geq$), le ($\leq$)
- regex vzory

Enum hodnoty

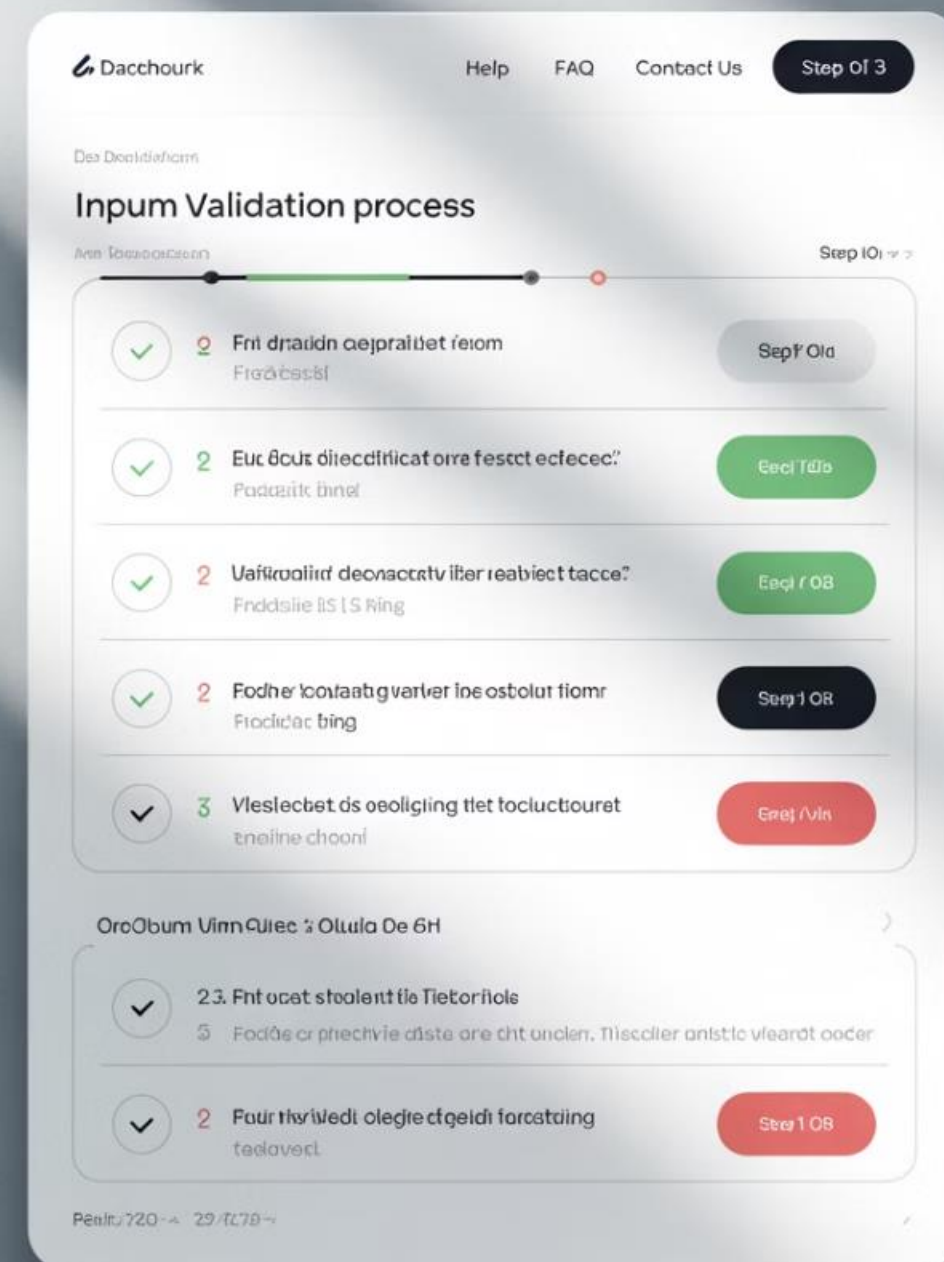
Obmedzenie na vopred definované hodnoty

- Výber z možností
- Automatické zobrazenie v dokumentácii

Vlastné validátory

Komplexná validácia pomocou @validator dekorátora

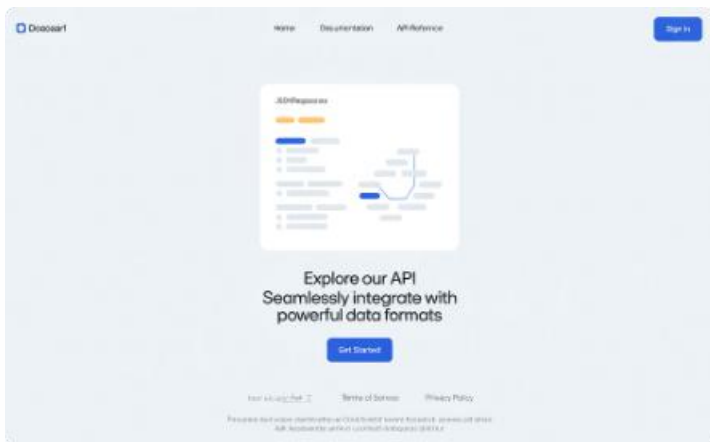
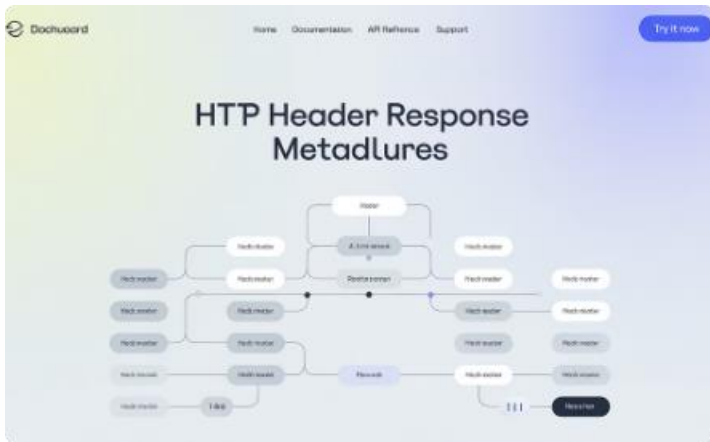
- Vzťahy medzi poliami
- Zložité podmienky



Nastavenie HTTP status kódu odpovede (200, 201, 404, atď.)

Pridanie vlastných HTTP hlavičiek do odpovede

Formátovanie obsahu odpovede (JSON, HTML, súbory)



Response models



Definícia výstupného modelu

Vytvorenie Pydantic modelu pre odpoved'



Filtrovanie dát

Automatické odstránenie nežiaducich polí



Bezpečnosť

Ochrana pred únikom citlivých údajov

Oddelenie modelov

Vstupný model

```
class UserInput(BaseModel):  
    name: str  
    password: str  
    email: str
```

Vstupné modely definujú údaje, ktoré API prijíma od klientov a zabezpečujú validáciu dát.

Výstupný model

```
class UserOutput(BaseModel):  
    name: str  
    email: str  
    id: int
```

Citlivé údaje ako heslo sa nevrátia klientovi.

Výstupné modely kontrolujú dáta odosielané z API a chránia pred únikom citlivých informácií.

Poznámka: Oddelenie vstupných a výstupných modelov je základom bezpečnej API architektúry a umožňuje lepšiu kontrolu nad dátovými tokmi.

Úlohy

Požiadavky

1. Čo sú to HTTP metódy, status kódy, endpoint routing?
2. Ako sa používajú GET, POST...?
3. Ako sa pracujeme s chybami?
4. Ako sa pracuje s pydantic?
5. Ako sa nastavuje PyCharm pre FastAPI?
6. Ako vytvoriť a testovať aplikáciu FastAPI Python + HTML + JS?

FastAPI HTTP Methods

GET

POST

PUT

PATCH

DELETE

Ako pracovať s JSON Dátami

AKREDITOVANÝ KURZ



FastAPI





Čo sa naučíme?

1. Ako sa ukladá model v Keras?
2. Ako sa načíta uložený model?
3. Aký je rozdiel medzi `save()` a `save_weights()`?
4. Ako sa vykonáva predikcia na nových dátach?
5. Na čo si dať pozor pri predikcii nových vstupov?
6. Ako sa využívajú uložené modely v praxi?
7. Profit

Práca s JSON dátami

Automatická konverzia

FastAPI automaticky konvertuje Python objekty na JSON. Nie je potrebné manuálne serializovať dáta pred odoslaním odpovede klientovi.



Obojsmerná transformácia

Podporuje konverziu camelCase <-> snake_case. Toto umožňuje používať Python konvencie v kóde a JavaScript konvencie na frontende.



Vnorené štruktúry

Spracovanie komplexných JSON objektov pomocou vnorených modelov. Vnorené Pydantic modely umožňujú reprezentovať zložité dátové štruktúry so zachovaním validácie.



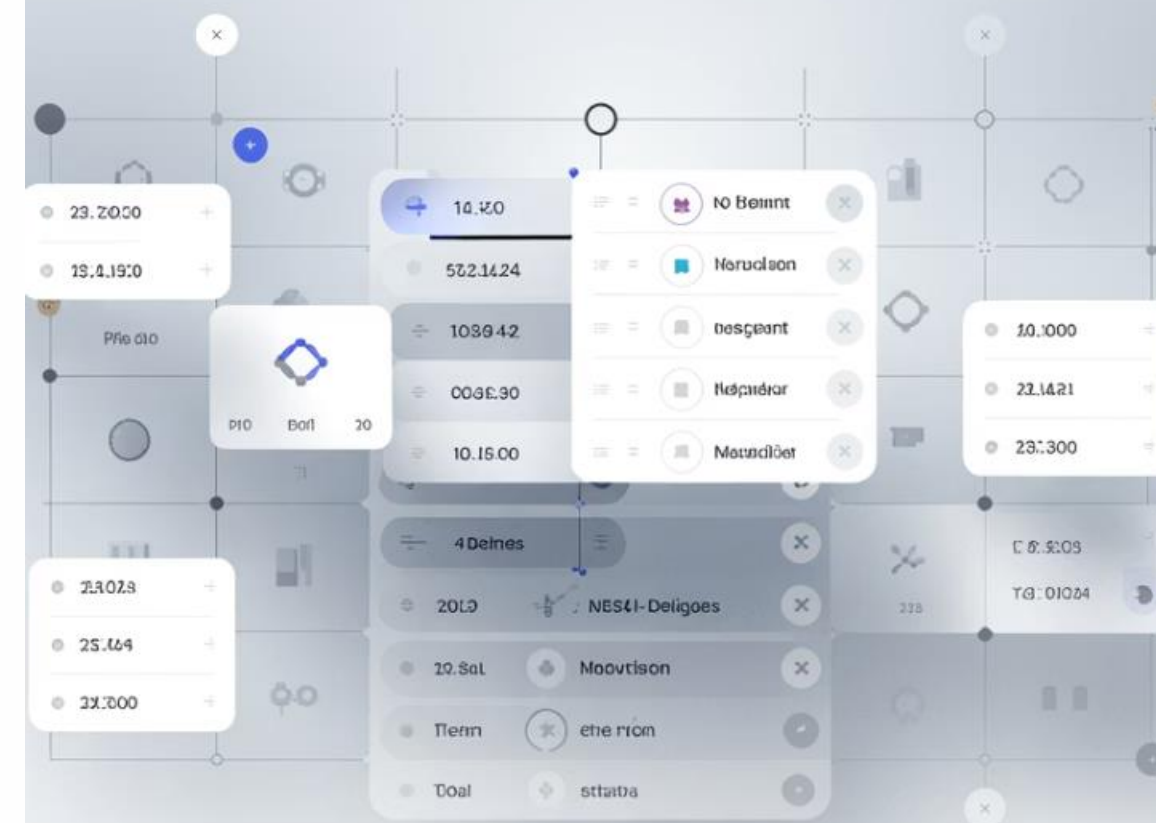
Validácia

Kontrola štruktúry a typov v JSON dátach. Pydantic automaticky generuje chybové hlásenia s presným popisom problému pri nesprávnom formáte vstupných dát.

Jsonio: JSON data structures

Učte si o JSON dátach, ktoré môžete použiť v Pythone a JavaScripte.

Explore Data



Spracovanie query parametrov

Definícia

```
@app.get("/produkty/") # Definícia endpointu
def zoznam_produkтов(
    kategoria: str = Query(...), # Povinný
    parameter
    cena_do: int = Query(default=100) #
    Nepovinný parameter
):
    # Vrátene JSON odpovede
    return {
        "kategoria": kategoria,
        "cena_max": cena_do
    }
```

Vysvetlenie

- ... znamená povinný parameter
- default=100 nastaví predvolenú hodnotu
- Query umožňuje kontrolovať dĺžku, regex, min/max
- Typová anotácia (str, int) zabezpečuje automatickú validáciu
- Query parametre sú vždy voliteľné v HTTP, ale FastAPI ich môže označiť ako povinné

Príklad URL: /produkty/?kategoria=elektronika&cena_do=500

Poznámka: Query parametre sa používajú na filtrovanie, stránkovanie a triedenie dát

Práca s path parametrami

1

Definícia cesty

```
@app.get("/pouzivatel/{pouzivatel_id}")
```

Poznámka: Časť v zložených zátvorkách {pouzivatel_id} predstavuje dynamický parameter, ktorý sa zachytí a pošle do funkcie.

2

Funkcia s parametrom

```
def info(pouzivatel_id: int):
```

Poznámka: Parameter má anotáciu typu int, čo zabezpečí automatickú konverziu a validáciu. Ak používateľ zadá nečíselnú hodnotu, automaticky sa vráti chyba.

3

Telo funkcie

```
return {"id": pouzivatel_id}
```

Poznámka: Funkcia vracia JSON objekt, ktorý obsahuje zachytený parameter. FastAPI automaticky skonvertuje Python slovník na JSON.

4

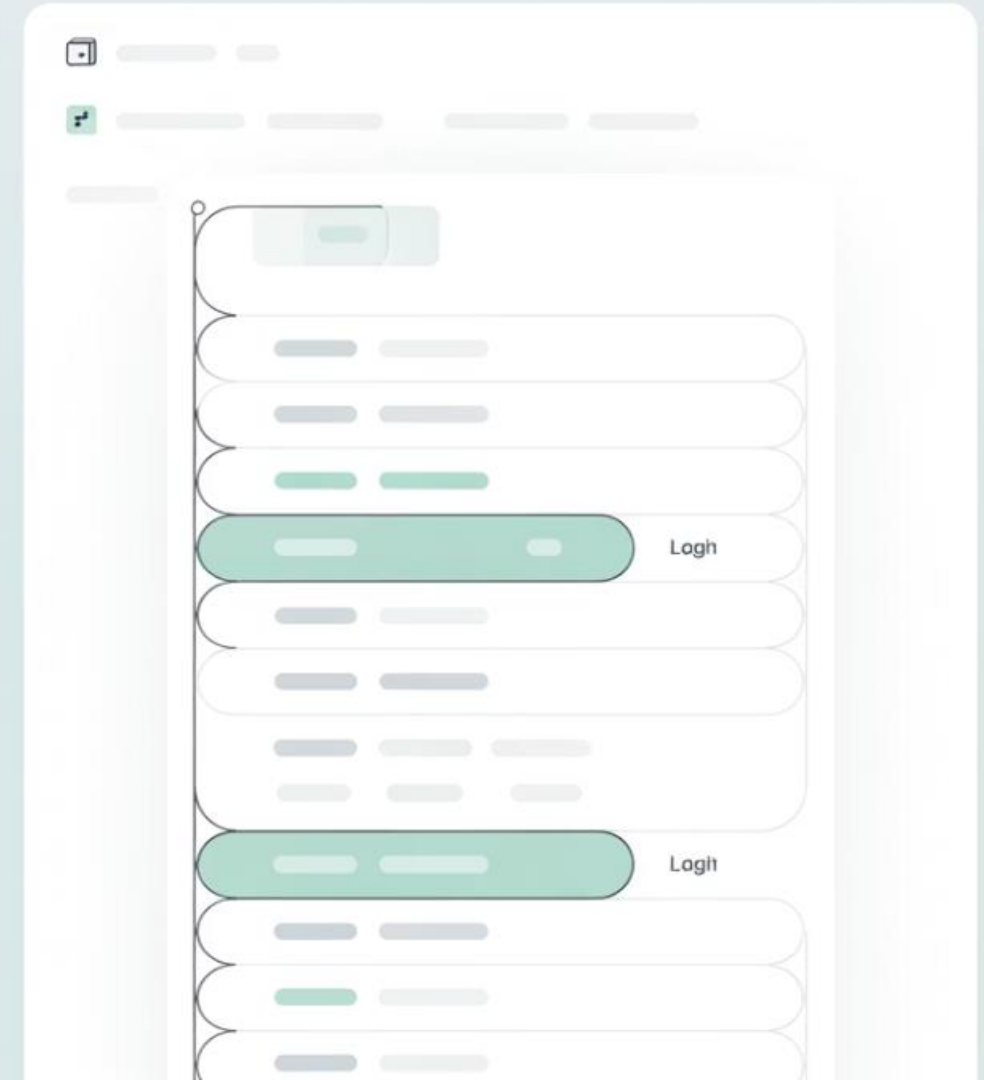
Výsledok

Pri požiadavke na /pouzivatel/42 vráti {"id": 42}

Poznámka: Path parametre sú ideálne pre identifikátory, ID položiek alebo akékoľvek hodnoty, ktoré sú súčasťou samotnej URL cesty.

Understanding your API

Discover our API >



Header a Cookie spracovanie

Poznámka: HTTP hlavičky a cookies sú kľúčové pre komunikáciu medzi klientom a serverom

Kód

```
from fastapi import Header, Cookie # Importujeme potrebné funkcie

@app.get("/info") # Endpoint pre získanie informácií
def info(
    agent: str = Header(default="neznámy"), # Poznámka: Získa User-Agent hlavičku
    session_id: str = Cookie(default="") # Poznámka: Získa session_id cookie
):
    # Poznámka: Vrátime informácie ako JSON odpoveď
    return {
        "user-agent": agent,
        "session": session_id
    }
```

Poznámka: Parametre funkcie automaticky extrahujú hodnoty z požiadavky

Poznámka: Tieto funkcie zjednodušujú prácu s HTTP komunikáciou bez potreby manuálneho spracovania požiadaviek

Vysvetlenie

- **Header** číta HTTP hlavičky (napr. User-Agent) *Poznámka: Hlavičky obsahujú metadáta o požiadavke*
- **Cookie** číta cookies poslané klientom *Poznámka: Cookies udržiavajú stav medzi požiadavkami*
- Predvolené hodnoty pre prípad, že údaje chýbajú *Poznámka: Zvyšuje odolnosť API*
- Možnosť validácie rovnako ako pri iných parametroch *Poznámka: Zabezpečuje správny formát údajov*

Poznámka: FastAPI automaticky generuje dokumentáciu aj pre tieto parametre



Username

Password

Request body cez Pydantic model

1

Model

```
class Prihlasenie(BaseModel):  
    meno: str  
    heslo: str
```

Definuje štruktúru dát a automaticky validuje vstupné dáta

2

Endpoint

```
@app.post("/login/")  
def prihlasenie(data: Prihlasenie):
```

FastAPI automaticky deserializuje JSON na Pydantic objekt

3

Spracovanie

```
return {  
    "status": f"Používateľ  
{data.meno} prihlásený"  
}
```

Pristupujeme k validovaným dátam cez atribúty objektu

Pydantic modely poskytujú automatickú validáciu, parsovanie a dokumentáciu API

Oddelenie vstupného a výstupného modelu

Bezpečnostný pattern na ochranu citlivých údajov

Vstupný model

Obsahuje všetky potrebné údaje vrátane citlivých

Poznámka: Napríklad heslo, osobné údaje alebo iné dôverné informácie

Výstupný model

Obsahuje len bezpečné údaje pre klienta

Poznámka: Zabezpečuje, že citlivé údaje neopustia server a nebudú vystavené v odpovedi API



Spracovanie

Biznis logika pracuje so všetkými údajmi

Poznámka: V tejto fáze systém môže údaje validovať, ukladať do databázy alebo ďalej spracovávať

Filtrovanie

Odstránenie citlivých údajov pred odpoveďou

Poznámka: Odstránenie hesiel, tokenov a iných informácií, ktoré klient nepotrebuje vidieť

Tento vzor je kľúčový pre dodržiavanie bezpečnostných štandardov a ochranu osobných údajov používateľov.

Príklad oddelenia modelov

```
class pouzivatelInput(BaseModel):
    # Vstupný model obsahujúci aj citlivé dáta
    meno: str
    heslo: str # Citlivý údaj

class pouzivatelOutput(BaseModel):
    # Výstupný model bez citlivých údajov
    meno: str # Iba verejné údaje

@app.post("/registracia/",
          response_model=pouzivatelOutput) #
Definícia výstupného modelu
def registruj(data: pouzivatelInput):
    # Funkcia prijíma kompletný vstupný model
    # Ale vráti iba údaje podľa výstupného modelu
    return pouzivatelOutput(meno=data.meno)
```

Vysvetlenie

- pouzivatelInput prijíma aj citlivé údaje
- pouzivatelOutput filtruje odpoveď
- Heslo sa nevracia späť klientovi
- response_model zabezpečuje automatické filtrovanie

Úlohy JSON

1. Ako sa ukladá model v Keras?
2. Ako sa načíta uložený model?
3. Aký je rozdiel medzi `save()` a `save_weights()`?
4. Ako sa vykonáva predikcia na nových dátach?
5. Na čo si dať pozor pri predikcii nových vstupov?
6. Ako sa využívajú uložené modely v praxi?



FastAPI

JSON

```
{  "name": "Alice",  
  "age" 30  
}
```

Ako Validovať Vstupy vo FastAPI?

AKREDITOVANÝ KURZ

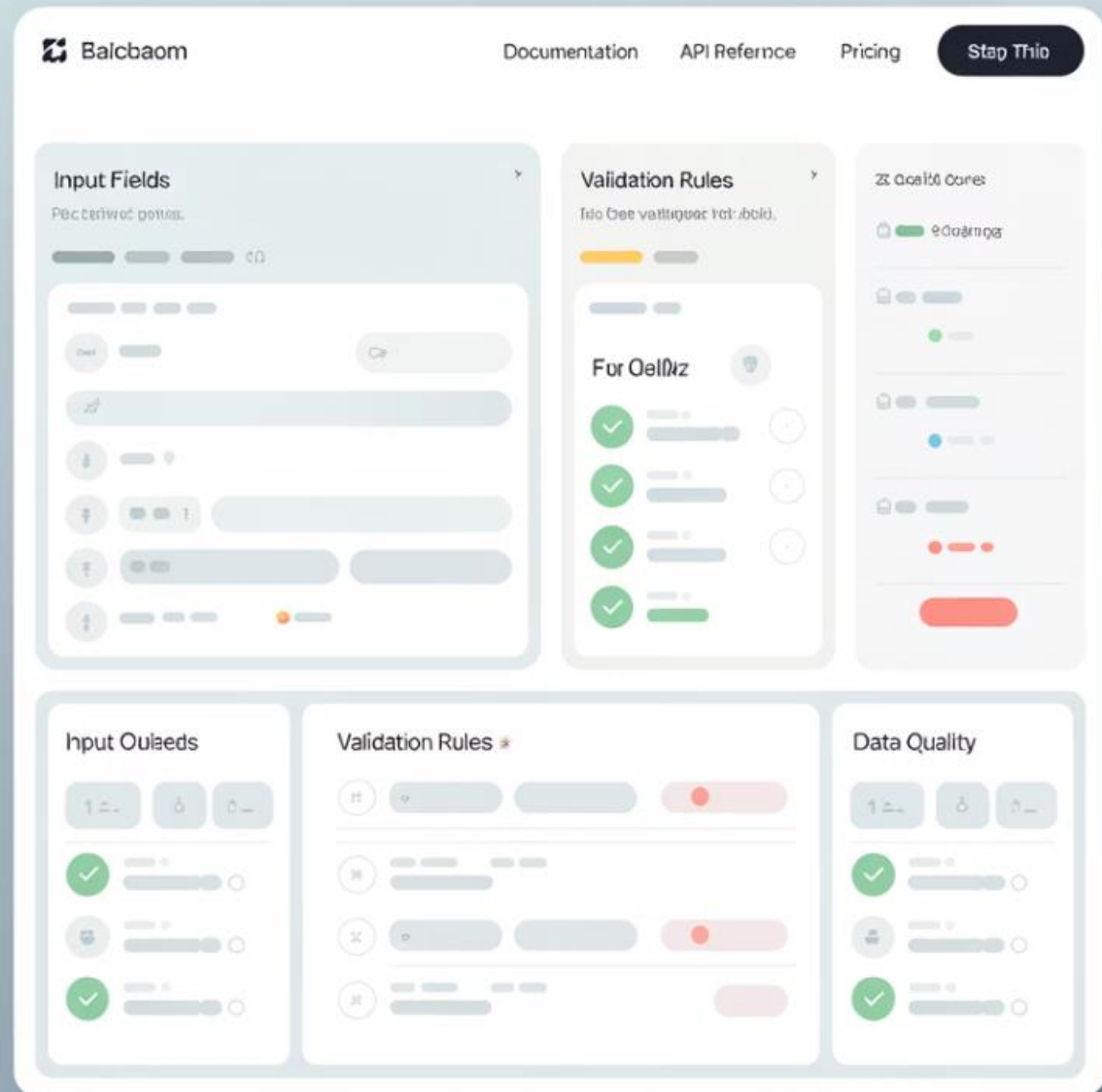




Čo sa naučíme?

1. Čo je overfitting a ako mu predchádzať?
2. Na čo slúži technika Dropout?
3. Ako funguje EarlyStopping?
4. Ako zlepšiť presnosť modelu?
5. Čo sú to metriky a ako sa používajú pri optimalizácii?
6. Na čo slúži ModelCheckpoint a ako pomáha optimalizácii?
7. Profit

Validácia vstupov



Start Free Trial

Dĺžka reťazcov

Kontrola minimálnej a maximálnej dĺžky textových polí.

`min_length=10, max_length=300`

Poznámka: Zabraňuje príliš krátkym alebo príliš dlhým vstupom, čím zlepšuje používateľskú skúsenosť a predchádza zneužitiu.

Číselné rozsahy

Obmedzenie hodnôt číselných polí.

`ge=0, le=150`

Poznámka: Parametre ge (greater or equal) a le (less or equal) definujú povolený rozsah hodnôt, čím zabraňujú neplatným hodnotám.

Regex vzory

Kontrola formátu pomocou regulárnych výrazov.

`regex=r'^\\S+@\\S+\\.\\S+$'`

Poznámka: Regulárne výrazy umožňujú komplexnú validáciu formátu, napríklad správny tvar e-mailovej adresy ako je uvedené v príklade.

Príklad validácie vstupov

```
class Kontakt(BaseModel):
    email: str = Field(
        ...,
        regex=r'^\\S+@\\S+\\.\\S+$'
    )
    sprava: str = Field(
        ...,
        min_length=10,
        max_length=300
    )

@app.post("/kontakt/")
def odoslat(k: Kontakt):
    return {"stav": "Odoslané", "email": k.email}
```

Vysvetlenie

- Field(..., ...) slúži na pokročilú validáciu
- Regex preverí správny formát e-mailu
- Kontrola dĺžky správy (10-300 znakov)
- Pri nevyhovujúcich dátach vráti API kód 422

Vlastné status kódy a hlavičky



Status kódy

Nastavenie HTTP kódu odpovede
(200, 201, 404, atď.)



Hlavičky

Pridanie vlastných HTTP hlavičiek
do odpovede



Obsah

Vlastný formát a štruktúra odpovede

```
from fastapi.responses import JSONResponse # Import triedy pre
vlastnú JSON odpoveď

@app.get("/special/") # Definícia endpointu na ceste /special/
def vlastna_odpoved():
    # Vytvorenie vlastnej odpovede s upraveným status kódom,
    # obsahom a hlavičkami
    return JSONResponse(
        status_code=202, # Nastavenie kódu 202 (Accepted)
        content={"msg": "Spracúva sa"}, # Vlastný obsah odpovede
        headers={"X-Info": "Čakajte prosím"} # Vlastná HTTP hlavička
    )
```

Práca s List a Optional typmi

```
from typing import List, Optional

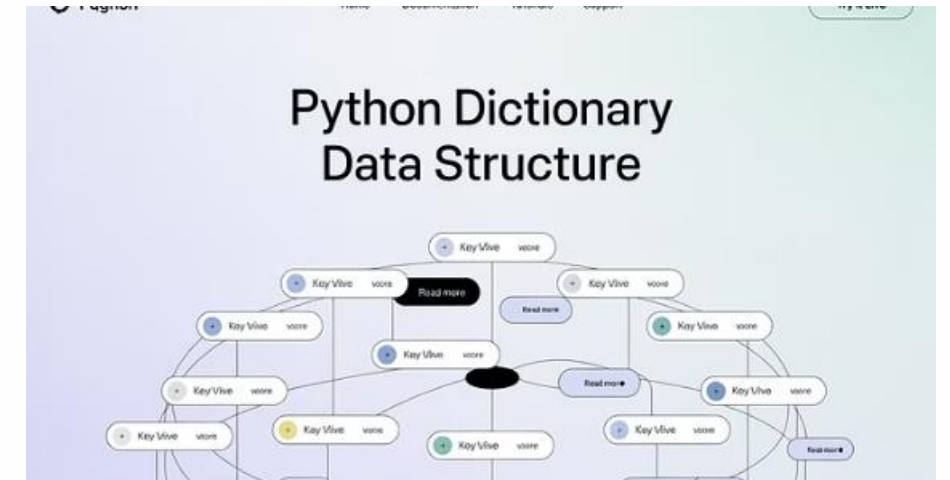
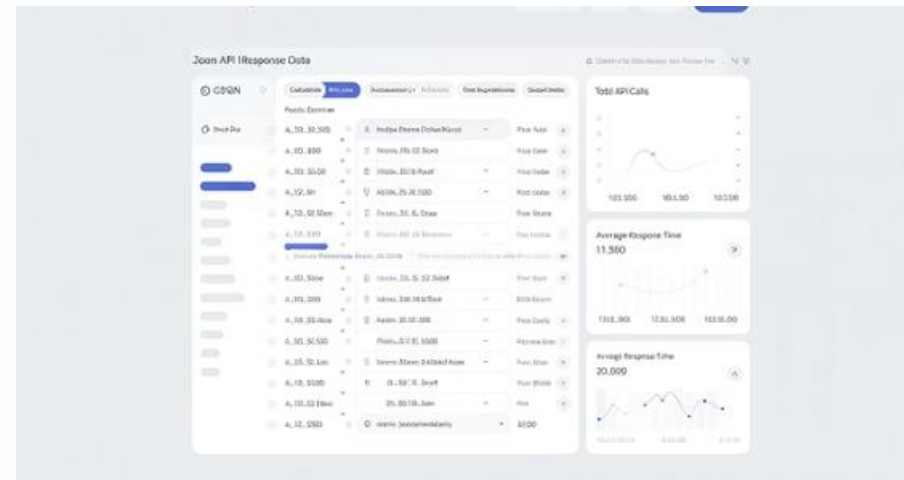
@app.get("/vyhladavanie/")
def vyhladaj(
    tagy: Optional[List[str]] = Query(None)
):
    return {"tagy": tagy}
```

Vysvetlenie

- Optional označuje nepovinný parameter
- List[str] umožňuje poslať viacero hodnôt
- Query(None) nastaví predvolenú hodnotu None
- Podporuje opakujúce sa parametre v URL

Príklad URL: /vyhladavanie/?tagy=AI&tagy=Python&tagy=Data

Odoslanie JSON zo slovníka



```
@app.post("/json/")
def json_input(data: dict):
    return {"priate": data}
```

Jednoduchý spôsob pre rýchle prototypovanie bez potreby definovať model.



Region

Validácia pomocou Enum

Definícia Enum

1

```
from enum import Enum

class Status(str, Enum):
    aktivny = "aktivny"
    neaktivny = "neaktivny"
    pozastaveny = "pozastaveny"
```

Poznámka: Enum v Pythone umožňuje definovať konštanty s pevnými hodnotami. Dedenie od str zabezpečuje kompatibilitu s FastAPI.

Použitie v endpointe

2

```
@app.get("/pouzivatelia/")
def filtrovat(status: Status):
    return {"stav": status}
```

Poznámka: FastAPI automaticky validuje hodnotu parametra a vyhodí chybu, ak hodnota nie je jednou z definovaných v Enum triede.

Výhody

3

Obmedzenie hodnôt na vopred definovaný výber

Swagger UI zobrazí rozbaľovací zoznam možností

Poznámka: Použitie Enum zvyšuje bezpečnosť a čitateľnosť kódu. Centralizuje definíciu povolených hodnôt na jednom mieste.

Asynchrónne API



Asynchrónne API umožňuje serveru spracovávať viacero požiadaviek paralelne bez blokovania. Toto je kľúčové pre výkonné aplikácie s vysokou priepustnosťou.

Na rozdiel od synchrónneho spracovania, asynchrónny prístup neblokuje vlákna počas čakania na I/O operácie, čo významne zlepšuje škálovateľnosť systému.

V jazyku Python sa asynchrónne funkcie označujú kľúčovým slovom "async" a využívajú príkazy "await" na neblokujúce čakanie na výsledky operácií.

Asynchrónny endpoint

```
import asyncio # Import knižnice pre asynchrónne operácie
from fastapi import FastAPI # Import FastAPI frameworku

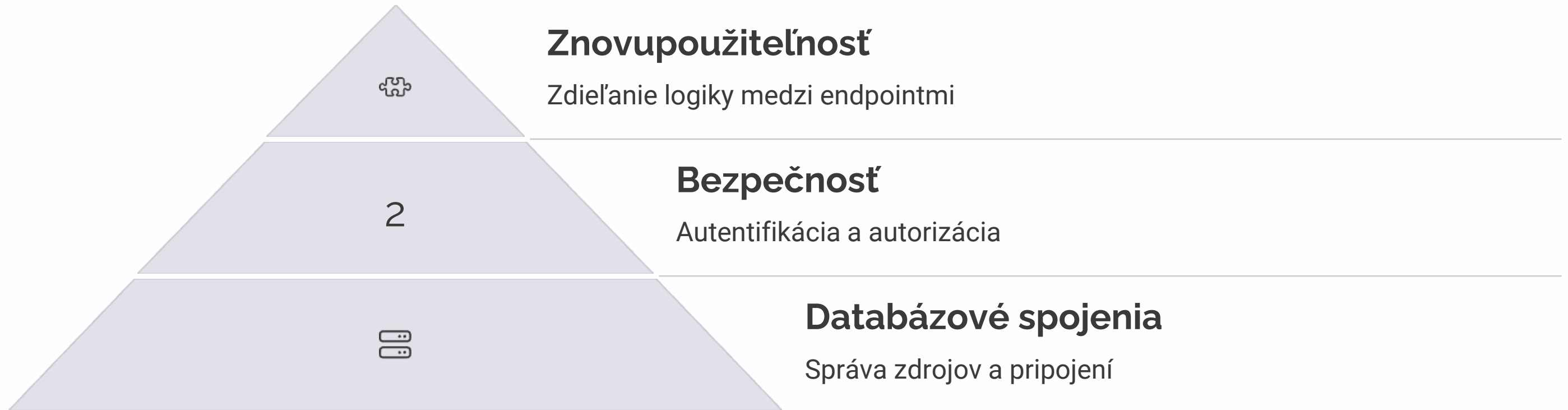
app = FastAPI() # Vytvorenie aplikácie

@app.get("/async-demo/") # Definícia cesty endpointu
async def async_endpoint():
    # Simulácia asynchrónnej operácie (napr. čakanie na DB)
    await asyncio.sleep(1) # Počkať 1 sekundu bez
    blokovania
    return {"message": "Dokončené"} # Vrátenie JSON
odpovede
```

Výhody

- Neblokujúce spracovanie požiadaviek
- Vyššia priepustnosť servera
- Efektívne využitie zdrojov
- Ideálne pre I/O náročné operácie
- Kompatibilita s asyncio ekosystémom

Závislosti (Dependencies)



Poznámky k závislostiach:

- **Znovupoužitelnost** - Závislosti umožňujú extrahovať spoločnú logiku do samostatných funkcií, ktoré možno používať v rôznych koncových bodoch. To zlepšuje udržiavateľnosť kódu a znižuje duplicitu.
- **Bezpečnosť** - Závislosti sú ideálne pre implementáciu bezpečnostných mechanizmov, ako je overenie tokenov, kontrola práv užívateľov alebo ochrana pred neoprávneným prístupom k API.
- **Databázové spojenia** - Pomocou závislostí možno efektívne spravovať pripojenia k databáze, zabezpečiť správne uvoľnenie zdrojov a implementovať pokročilé funkcie ako connection pooling.

Príklad závislosti

```
from fastapi import Depends, FastAPI

# Vytvorenie aplikácie
app = FastAPI()

# Funkcia závislosti, ktorá spracováva token
def get_query_token(token: str):
    # Jednoduchá funkcia, ktorá vracia token
    return token

# Definícia API endpointu s cestou "/items/"
@app.get("/items/")
async def read_items(token: str =
Depends(get_query_token)):
    # Použitie hodnoty z funkcie závislosti
    return {"token": token}
```

Vysvetlenie

- Depends injektuje výsledok funkcie `get_query_token`
- Funkcia závislosti sa vykoná pred hlavnou funkciou
- Výsledok sa predá ako parameter
- Umožňuje zdieľať logiku medzi viacerými endpointmi



Bezpečnosť a autentifikácia

OAuth2

Štandardný protokol pre autorizáciu.

- Access tokeny
- Refresh tokeny
- Scope obmedzenia

Poznámka: Široko používaný v enterprise aplikáciách. Vhodný pre single sign-on riešenia.

JWT

JSON Web Tokens pre bezpečný prenos informácií.

- Podpísané tokeny
- Claim-based autorizácia
- Stateless autentifikácia

Poznámka: Jednoduchá integrácia s FastAPI. Pozor na veľkosť tokenov a bezpečnostné riziká.

API kľúče

Jednoduché riešenie pre menej kritické API.

- Jednoduchá implementácia
- Limitovanie prístupu
- Sledovanie používania

Poznámka: Vhodné pre verejné API s nízkou úrovňou zabezpečenia. Ľahko implementovateľné s FastAPI Depends.

Oauth2 Authorication

Orenalmehare tlutirizoot voov uzitib; apitolocatiy,
outhiricinsating autociatins sovrid soive;

[Learn more](#)

Authorization

Leer esint dopnloatinr
coppleatins ou oloeatins
auveratloiois hior,
pottoceteds teevring
oatvoo t bovies...

Scopes

Leer esint dopnloatinr
bepileatins oactpeltins
enres ootatvot, fvar,
pottoceteds teevring
oatvoo t bovies...

Tokens

Leer esint dopnloatinr
conleatins ou oloeatins
ndigst vaint atocetadit
pottoceteds teevring
oatvoo t bovies...

Tokens

Leer esint dopnloatinr
oeploatins ou oloeatins
tontopattvot; toatvot
pottoceteds teevring
oatvoo t bovies...

Scopes

Leer esint dopnloatinr
conleatins ou oloeatins
ohleratins, nassamint
pottoceteds teevring
oatvoo t bovies...

OAuth2 implementácia

Nastavenie závislostí

1

```
from fastapi.security import OAuth2PasswordBearer
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```

Táto časť importuje potrebné OAuth2 knižnice a inicializuje OAuth2 schému s definovaným endpointom pre získanie tokenu.

Použitie v endpointe

2

```
@app.get("/users/me")
async def read_users_me(token: str = Depends(oauth2_scheme)):
    return {"token": token}
```

Endpoint /users/me používa OAuth2 schému cez Depends na vyžiadanie a extrakciu tokenu z hlavičky požiadavky, čím zabezpečuje, že len autentifikovaní používatelia môžu pristupovať k údajom.

Overenie tokenu

3

```
def get_current_user(token: str = Depends(oauth2_scheme)):
    # Overenie a dekódovanie tokenu
    return user
```

Funkcia get_current_user dekóduje a overuje platnosť tokenu, extrahuje identitu používateľa a vracia objekt používateľa pre ďalšie operácie v aplikácii.

Middleware

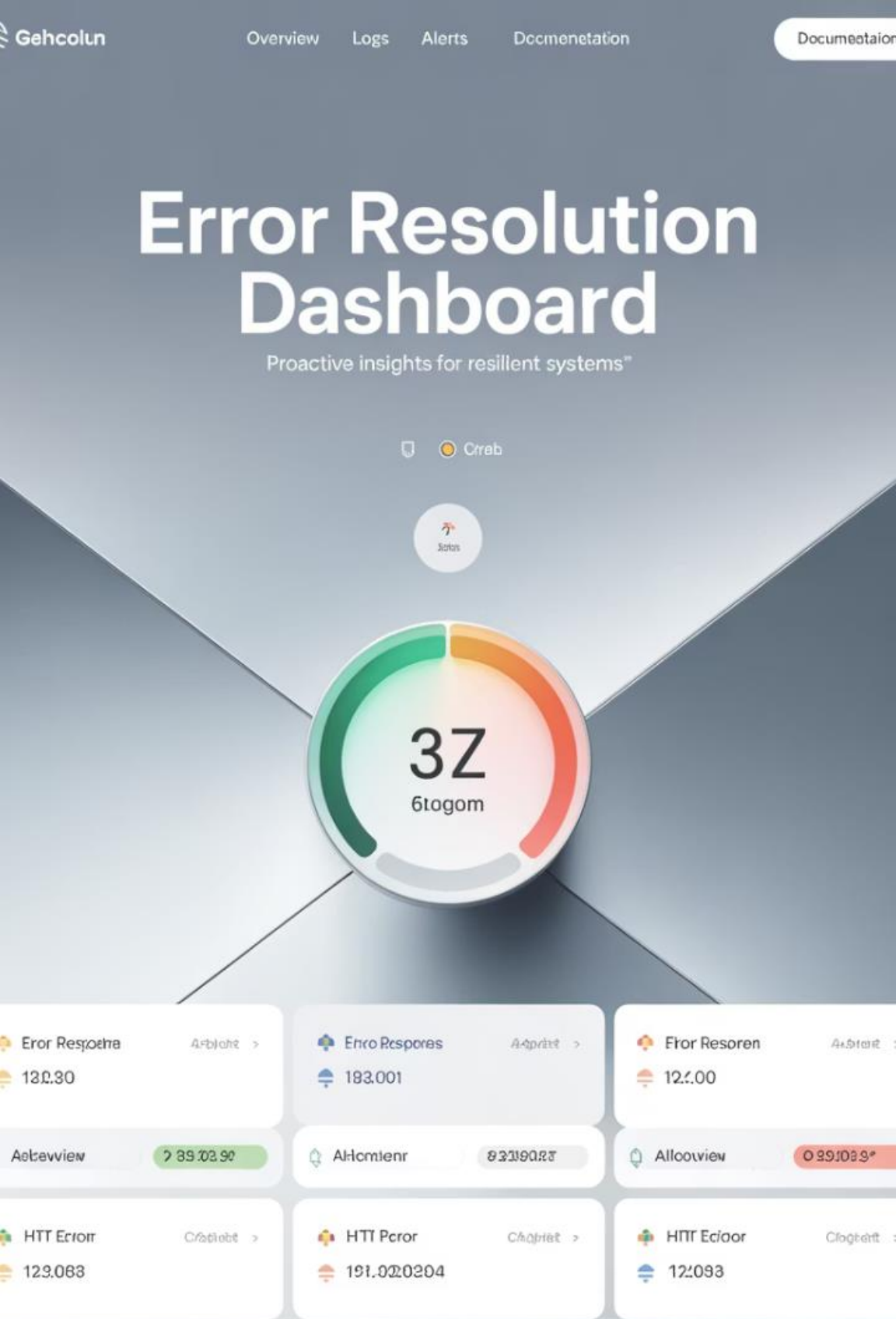


Príklad middleware

```
@app.middleware("http") # Dekorátor pre HTTP
middleware
async def add_process_time_header(request,
call_next):
    # Začiatok merania času
    start_time = time.time()
    # Volanie ďalšieho middlewaru alebo endpointu
    response = await call_next(request)
    # Výpočet času spracovania
    process_time = time.time() - start_time
    # Pridanie hlavičky s časom do odpovede
    response.headers["X-Process-Time"] =
        str(process_time)
    return response # Vrátenie upravenej odpovede
```

Použitie

- Meranie výkonu - sledovanie času spracovania požiadaviek
- Logovanie požiadaviek - zaznamenávanie prichádzajúcich HTTP dotazov
- Autentifikácia - overenie identity používateľa pred spracovaním
- CORS nastavenia - konfigurácia zdieľania zdrojov medzi rôznymi doménami
- Spracovanie chýb - jednotný systém zachytávania a spracovania výnimiek
- Kompresia odpovedí - zmenšenie veľkosti dát pre rýchlejší prenos



Spracovanie chýb

Definícia výnimky

```
from fastapi import HTTPException

def get_item(item_id: int):
    if item_id not in items:
        raise HTTPException(
            status_code=404,
            detail="Item not found"
        )
```

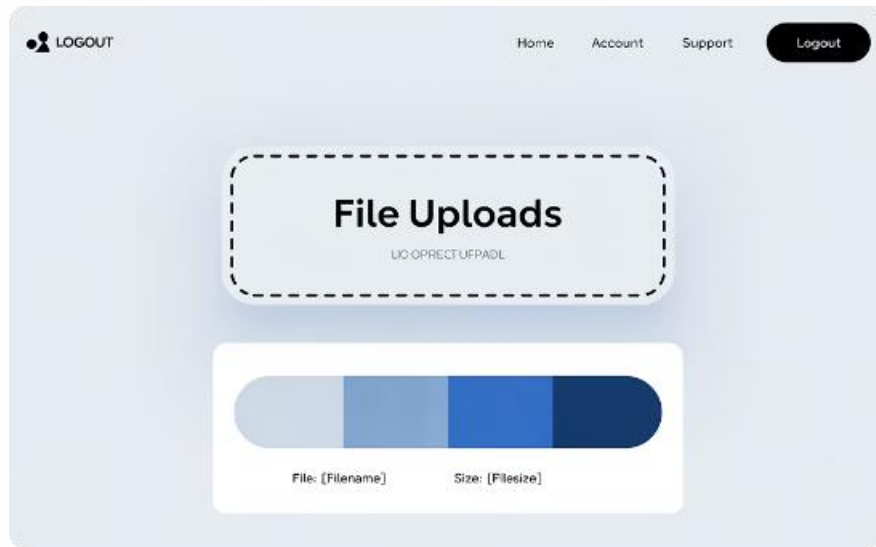
Vlastné chybové handlers

```
@app.exception_handler(UnicornException)
async def unicorn_exception_handler(request, exc):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name}"}
    )
```

Globálne zachytávanie

Možnosť zachytiť a spracovať všetky neošetrené výnimky.

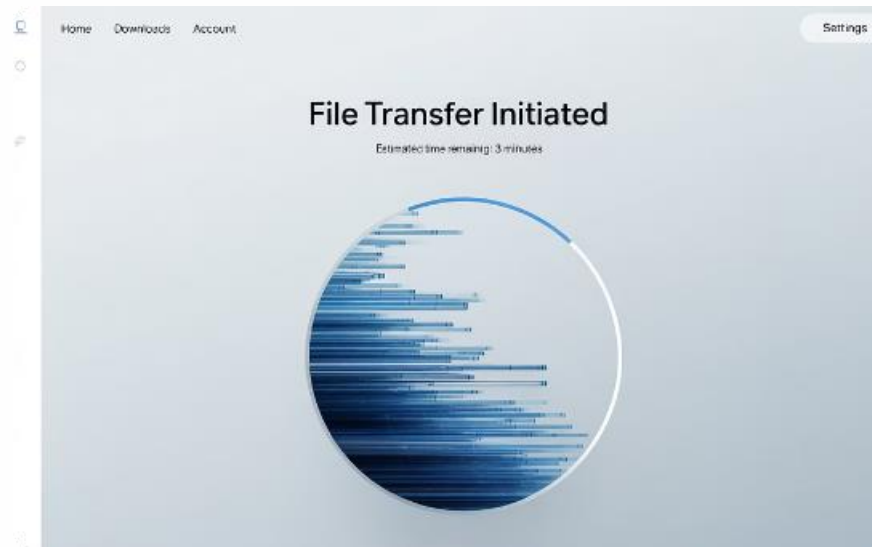
Práca so súbormi



Nahrávanie súborov

Spracovanie súborov odoslaných klientom pomocou `File` a `UploadFile`.

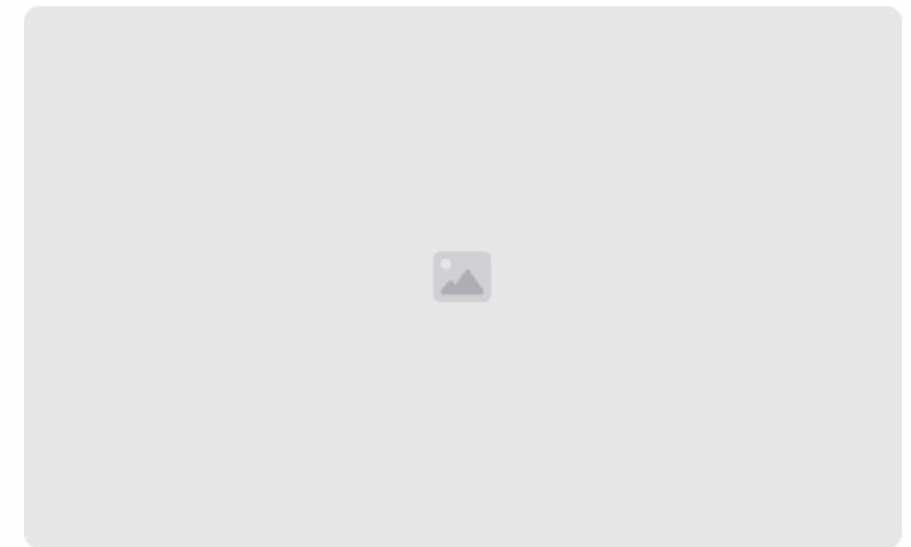
Poznámka: Použite `UploadFile` pre väčšie súbory a asynchrónne spracovanie. Podporuje rôzne formáty súborov vrátane obrázkov, dokumentov a binárnych dát.



Sťahovanie súborov

Odosielanie súborov klientovi pomocou `FileResponse`.

Poznámka: `FileResponse` automaticky nastaví správne hlavičky a podporuje streamovanie pre efektívny prenos veľkých súborov. Môžete špecifikovať `media_type` a `filename`.



Ukladanie súborov

Asynchrónna práca so súbormi pomocou `aiofiles`.

Poznámka: `Aiofiles` umožňuje non-blocking I/O operácie, čo zlepšuje výkon pri práci s viacerými súbormi súčasne. Ideálne pre high-load aplikácie.

Príklad nahrávania súborov

```
from fastapi import FastAPI, File, UploadFile # Importujeme potrebné
knižnice

app = FastAPI() # Vytvoríme inštanciu aplikácie

@app.post("/uploadfile/") # Definujeme POST endpoint pre nahrávanie
súborov
async def create_upload_file(
    file: UploadFile = File(...) # Parameter file typu UploadFile, ...
    # znamená povinný parameter
):
    contents = await file.read() # Asynchrónne načítanie obsahu súboru
    # Spracovanie obsahu súboru
    return { # Vrátime odpoveď s informáciami o súbore
        "filename": file.filename, # Vrátime názov súboru
        "content_type": file.content_type # Vrátime typ obsahu súboru
    }
```

Výhody UploadFile

- Spracovanie veľkých súborov - neuchováva celý súbor v pamäti
- Prístup k metadátam súboru - názov, MIME typ, veľkosť
- Asynchrónne čítanie obsahu - neblokuje hlavné vlákno
- Prístup k SpooledTemporaryFile - využíva dočasné úložisko
- Automatické uvoľnenie pamäte - súbor sa uzavrie po dokončení požiadavky

Testovanie FastAPI aplikácií

TestClient

Integrovaný klient pre testovanie API endpointov.

- Simulácia HTTP požiadaviek
- Overenie odpovedí
- Testovanie status kódov

Pytest

Populárny framework pre testovanie Python aplikácií.

- Fixture systém
- Parametrizované testy
- Pokrytie kódu

Asyncio testy

Testovanie asynchrónnych endpointov.

- pytest-asyncio
- async/await v testoch
- Mockovanie asynchrónnych závislostí



Elevate Your Code Quality

Automated outcomatone for coopniet tusting.
trecoliothes olulicort dnoct to creolshcor ucerine vtr
caring lro foosdng suphing tustolone tustict.



Test Automation

Leunheic leatortkono ot
veodiet ocrlotolonevot
piontiorotfz etlira
rate ardis tpoetitia.



Code Quality Assurance

Amnot set trestetion
olands to beot sming
ocroai an a'oretanne
peticall boietoti.



Software Testing Process

Levofitro robo hndrens
mtrictis for otocroalnet
arocootor oole otterloe
arocootor...

Príklad testovania

```
from fastapi.testclient import TestClient # Import
testovacieho klienta
from main import app # Import našej aplikácie

client = TestClient(app) # Vytvorenie testovacieho
klienta

def test_read_main(): # Testovacia funkcia pre hlavný
endpoint
    # Odoslanie GET požiadavky na koreňovú cestu
    response = client.get("/")
    # Overenie, že status kód je 200 (OK)
    assert response.status_code == 200
    # Overenie, že odpoveď obsahuje očakávaný JSON
    assert response.json() == {
        "message": "Hello, FastAPI"
    }
```

Pokročilé testovanie

- Mockovanie databázových spojení
- Testovanie autentifikácie
- Overenie validácie vstupov
- Testovanie chybových stavov
- Integračné testy s reálnymi závislosťami

Deployment



Nasadenie FastAPI aplikácií



Docker

Kontajnerizácia aplikácie pre konzistentné nasadenie v rôznych prostrediach.

Poznámka: Využite multi-stage builds pre menšie obrazy a Docker Compose pre orchestráciu viacerých služieb.



Cloud platformy

AWS, Google Cloud, Azure, Heroku a ďalšie cloudové služby.

Poznámka: Porovnajte cenové modely a dostupné služby pre optimálne nasadenie vašej aplikácie.



ASGI servery

Uvicorn, Hypercorn, Daphne pre produkčné nasadenie.

Poznámka: Uvicorn v kombinácii s Gunicorn je odporúčaná konfigurácia pre vysokú záťaž.



Nginx/Apache

Reverse proxy pre lepšiu škálovateľnosť a bezpečnosť.

Poznámka: Nakonfigurujte vyrovňavanie záťaže a SSL certifikáty pre bezpečnú komunikáciu.

Dockerfile pre FastAPI

```
FROM python:3.9
# Základný image s Pythonom 3.9

WORKDIR /app
# Nastavenie pracovného adresára v kontajneri

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Inštalácia závislostí pre rýchlejší build

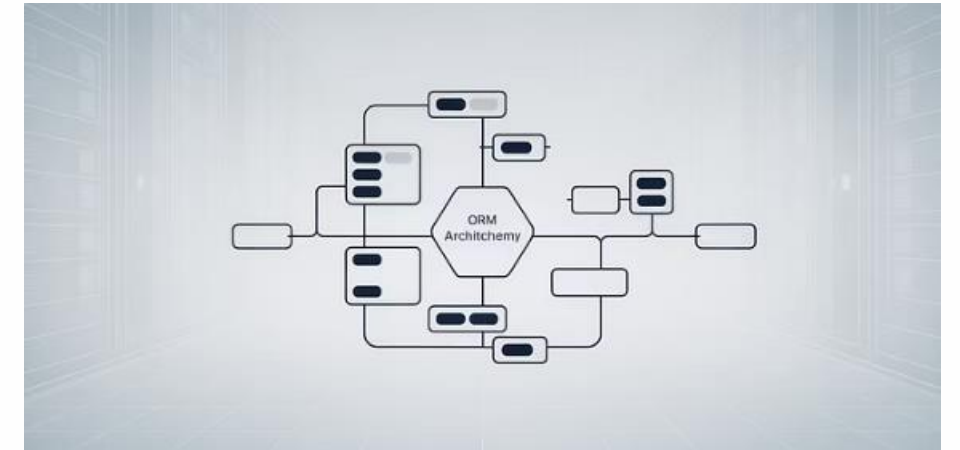
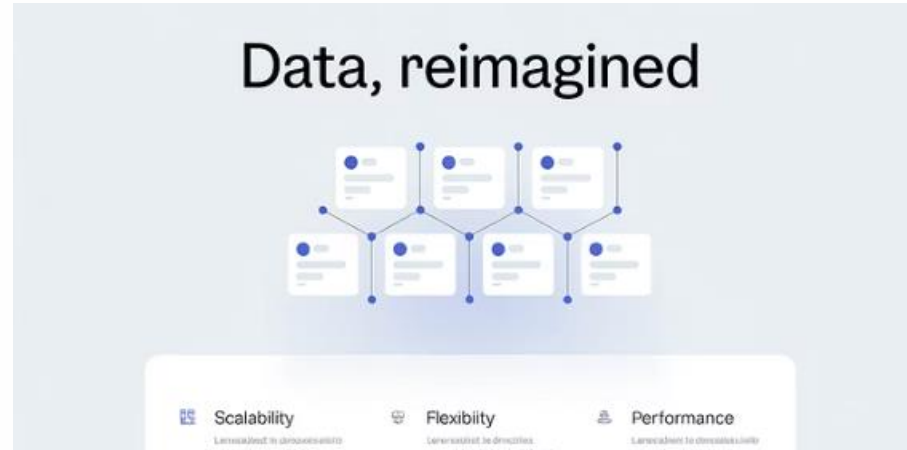
COPY . .
# Skopírovanie kódu aplikácie do kontajnera

CMD ["uvicorn", "main:app", "--host", "0.0.0.0",
"--port", "8000"]
# Spustenie aplikácie s Uvicorn serverom
```

Produkčné nasadenie

- Gunicorn ako process manager - *Správca procesov pre viacero workers*
- Uvicorn ako ASGI server - *Asynchrónny server pre FastAPI*
- Nginx ako reverse proxy - *Správa požiadaviek a load balancing*
- HTTPS s Let's Encrypt - *Bezplatné SSL certifikáty*
- Monitorovanie a logovanie - *Sledovanie výkonu a chýb*

Integrácia s databázami



SQLAlchemy

Populárny ORM pre relačné databázy.

Umožňuje objektovo-relačné mapovanie, podporuje rôzne SQL dialekty a poskytuje vysokú mieru abstrakcie. Jednoduchá integrácia s FastAPI pomocou Pydantic modelov.



MongoDB

Motor pre prácu s NoSQL databázami.

Flexibilné ukladanie dokumentov, škálovanie horizontálne a vertikálne, rýchle dotazovanie. V FastAPI možno použiť s knižnicou motor alebo pymongo pre asynchrónne operácie.

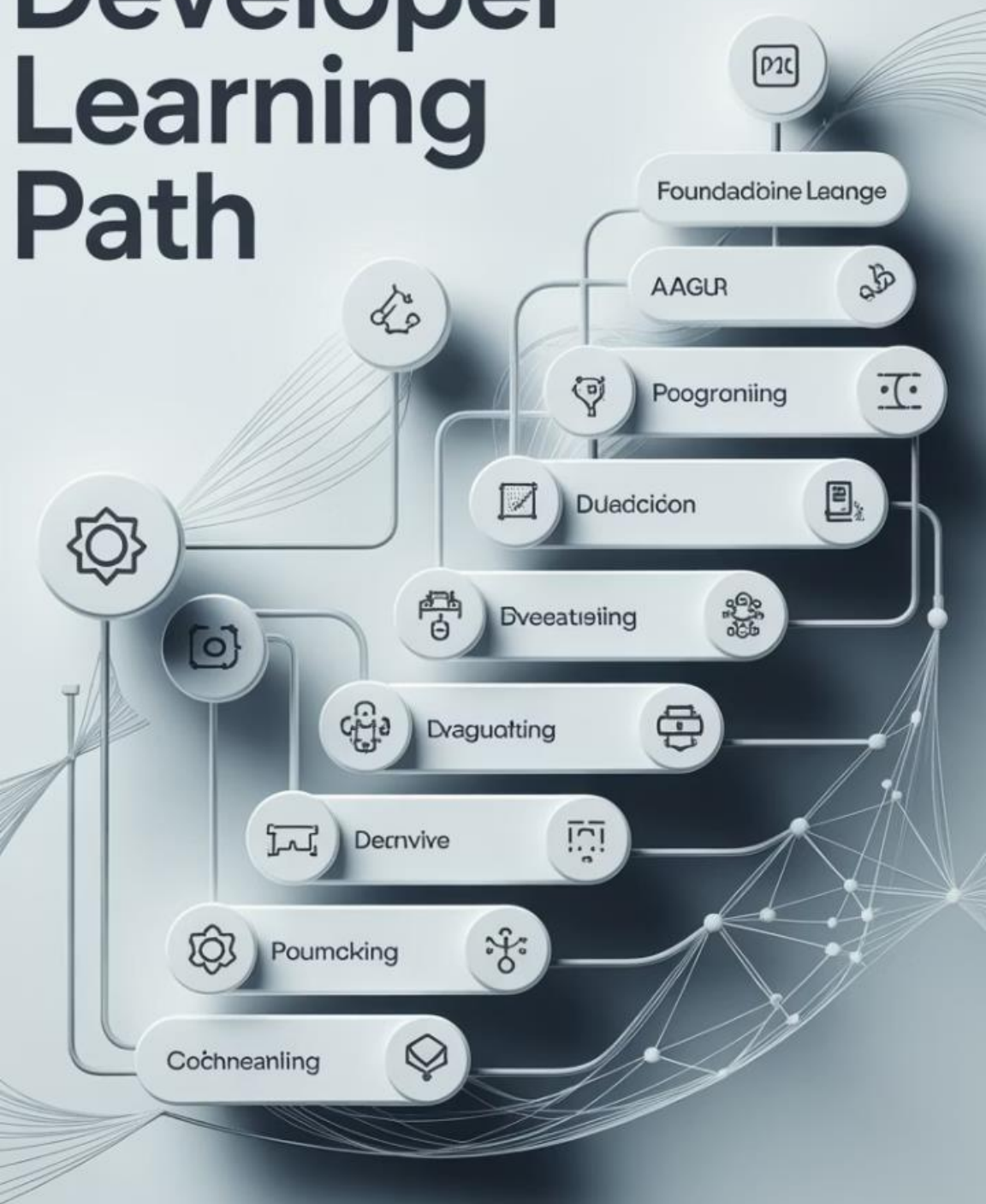


Tortoise ORM

Asynchrónny ORM pre FastAPI aplikácie.

Natívna podpora asyncio, jednoduchá syntax podobná Django ORM, schémy definované pomocou modelov. Ideálne pre vysokovýkonné asynchrónne API.

Developer Learning Path



Zhrnutie a ďalšie kroky

Základy FastAPI

Pochopenie základných konceptov, endpointov a validácie.

Poznámka: Začnite s oficiálnym tutoriálom a vytvorte jednoduchý CRUD endpoint.

Pokročilé funkcie

Asynchrónne API, závislosti, bezpečnosť a middleware.

Poznámka: Implementujte JWT autentifikáciu a vlastné middleware pre logovanie.

Reálne projekty

Integrácia s databázami, testovanie a nasadenie aplikácií.

Poznámka: Vyskúšajte dockerizáciu aplikácie a automatizované testy pomocou pytest.

Ďalšie štúdium

Oficiálna dokumentácia, open-source projekty, komunita.

Poznámka: Sledujte GitHub repozitár FastAPI a zapojte sa do diskusií na Discord kanáli.